

ChaosFormer: Hardware-Bound Edge Transformer Model Obfuscation for Intellectual Property Protection

PEICHUN HUA, School of Data Science, The Chinese University of Hong Kong, Shenzhen, P.R. China
YUE ZHENG*, School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, P.R. China

Building a production-level deep learning (DL) model requires substantial data, computational resources, and expert knowledge, making it a valuable asset vulnerable to intellectual property (IP) theft. Apart from violating commercial interests, the stolen model facilitates adversarial attacks and reduces the cost of building malicious applications. Thus, it is urgent today to protect DL models from leakage and theft. Existing solutions, such as watermarking and fingerprinting, provide only passive ownership verification after model theft. Instead, we explore an active protection method for edge-deployed transformer models. This paper presents ChaosFormer, a hardware-bound obfuscation framework that binds model executions to specific hardware using physical unclonable functions (PUF). We introduce a base permutation scheme for efficiency and a secure extension (ChaosFormer-S) incorporating diffusion for enhanced security. Under the stated production-runtime assumptions, a copied encrypted artifact executes correctly only with the authorized device and the final key derived from its PUF-rooted hierarchy. We provide a formal analysis of the chaotic permutation and prove Single-Point Indistinguishability for ChaosFormer-S. Experiments show that unauthorized execution collapses encrypted models from 77.1–86.4% ImageNet accuracy to near-random levels of 0.07–0.15%; under retraining attacks, the base scheme substantially reduces stolen-model utility, while ChaosFormer-S further lowers the retrained ViT-B performance below the same-architecture random initialization, e.g., 27.36% vs. 36.19% on ImageNet and 44.54% vs. 52.95% on DomainNet. In addition, our proposed method does not require additional retraining and incurs little inference overhead. This approach offers a low-cost, effective solution for the secure deployment of DL models at the edge. Our source code is publicly available at <https://github.com/sarendis56/ChaosFormer-IP-Protection>.

CCS Concepts: • **Security and privacy** → **Software and application security**; **Hardware-based security protocols**; • **Computing methodologies** → *Neural networks*.

Additional Key Words and Phrases: Intellectual Property Protection, Transformer Models, Physical Unclonable Functions, Adversarial Machine Learning, Format-Preserving Encryption

ACM Reference Format:

Peichun Hua and Yue Zheng. 2025. ChaosFormer: Hardware-Bound Edge Transformer Model Obfuscation for Intellectual Property Protection. *ACM Trans. Des. Autom. Electron. Syst.* 0, 0, Article 0 (January 2025), 45 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Corresponding author.

Authors' Contact Information: Peichun Hua, School of Data Science, The Chinese University of Hong Kong, Shenzhen, Shenzhen, P.R. China, peichunhua@link.cuhk.edu.cn; Yue Zheng, School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, Shenzhen, P.R. China, zhengyue@cuhk.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7309/2025/1-ART0

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Deep neural networks (DNNs) have become indispensable in many applications, driving remarkable achievements in computer vision, natural language processing, and more. Among them, *transformer* [119] has emerged as a predominant architecture across various tasks due to its superior performance and scalability. These models typically demand extensive data collection and massive computational resources for training, making well-tuned weights a valuable intellectual property (IP) asset.

Deploying transformers at the edge (e.g., in smartphones, IoT devices, or embedded systems) can bring substantial benefits, such as low latency, reduced cloud dependence, powerful functionality without internet connectivity, and enhanced user privacy. However, it also introduces critical security challenges. Once a transformer model is locally stored on a device, malicious actors may seek to *steal* and *misuse* its parameters [26, 50]. With access to these parameters, they could create harmful applications with minimal effort or launch adversarial attacks targeting the original model.

Although various IP protection strategies have been proposed for neural networks, ranging from watermarking [2, 47, 117] and fingerprinting [131, 144] to trusted execution environments (TEEs) [67, 142], they are often ill-suited for transformers deployed at the edge. Watermarking and fingerprinting provide only a *passive* ownership verification mechanism rather than proactively preventing model theft. For example, a systematic evaluation finds that surveyed watermarking schemes fail under adaptive removal attacks [75]; thus these methods only complement access control as another layer of defense. Meanwhile, TEE-based approaches [67, 142] introduce substantial runtime overhead and memory constraints, which are particularly problematic for large transformer models.

Consequently, there is a critical need for a *lightweight* and *proactive* IP protection method that can safeguard *on-device* transformers without requiring retraining or complicated hardware support. Specifically, we seek a solution that: (1) ensures only authorized hardware can correctly run the model (Sec. 5.1); (2) incurs minimal inference overhead (Sec. 5.3); (3) does not require retraining the original model (Sec. 4.2); (4) preserves authorized-model accuracy (Sec. 5.1); (5) maintains platform independence by operating solely on model weights without using framework-specific features (Sec. 4.2); and (6), through the secure extension, remains robust even if attackers have partial access to the training data, full knowledge of the network architecture, and access to public pre-trained reference models (Sec. 5.2.1).

In this paper, we propose ChaosFormer, a hardware-bound encryption framework designed for the proactive IP protection of transformers. ChaosFormer *binds* the transformer model's execution to a specific hardware device in order to prevent unauthorized usage — even if an attacker gains white-box access to the stored parameters. Our approach leverages (1) Physical Unclonable Functions (PUF) to bind the model to the target device. Each target device reconstructs a device root K_{master} from its PUF, while HKDF binds the final encryption key to the device, owner, model, version, and scheme. The vendor protects its enrollment record, and the device reconstructs the root and descendant keys only inside the authorized key path; and (2) reversible weight obfuscation combined with highly efficient permutation to effectively scramble the weights in a reversible yet highly disruptive manner. We proffer two schemes: the base ChaosFormer uses efficient permutation for low latency, while ChaosFormer-S adds a diffusion step to ensure computational indistinguishability from uniform under the PRF and key-secrecy assumptions. The correct final model-version-scheme key is indispensable for reconstructing the original layer weights on the authorized device. Our main contributions are as follows:

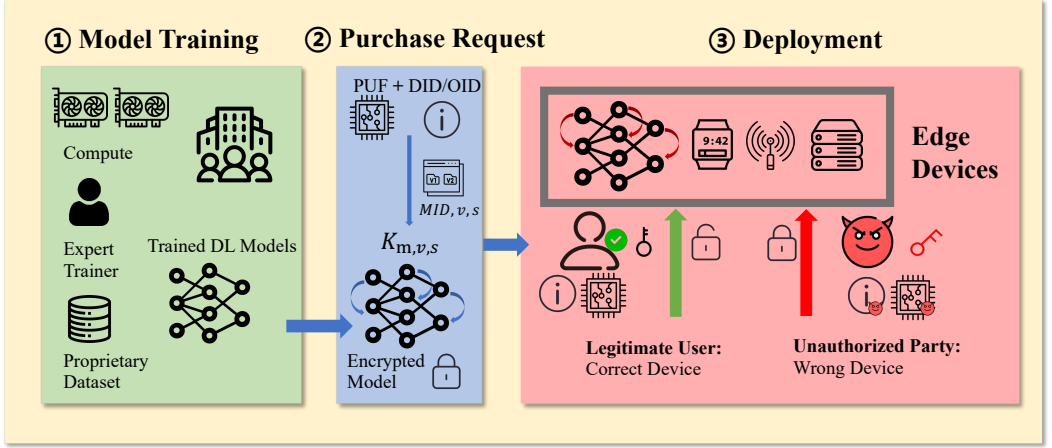


Fig. 1. Overview of our proposed framework, which consists of three stages. (1) Model Training: The trainer trains the model using proprietary datasets and compute resources. (2) Purchase Request: The model is encrypted and bound to the purchaser’s device using a PUF-rooted key hierarchy, ensuring that the model can only be deployed on authorized hardware. (3) Deployment: The model remains secure against offline checkpoint extraction from storage, distribution channels, or unauthorized hardware, as unauthorized decryption is infeasible without the corresponding final model-version-scheme key.

- A novel proactive IP protection framework that locks transformer models so that they can execute correctly only on authorized devices with the valid model-version-scheme keys derived from their PUF-rooted key hierarchies.
- We provide formal security guarantees, proving that our permutation exhibits chaotic behavior (avalanche effect) and that ChaosFormer-S satisfies Single-Point Indistinguishability (SPI).
- Extensive experiments show substantial recovery degradation for the base scheme and that, under the extended public-checkpoint threat model, a stolen ChaosFormer-S model provides no utility benefit over training the same architecture from scratch.
- The inference overhead is minimal, and our obfuscation process applies to various transformer architectures, making it practical for real-world edge deployments.

The rest of the paper is organized as follows. Sec. 2 introduces background on transformer architectures, Arnold’s Cat Map, and PUF. Sec. 3 details our threat model. Our IP protection framework is presented in Sec. 4. Sec. 5 describes our experimental evaluation and comparisons. We discuss related work in Sec. 6 and conclude in Sec. 7.

2 Preliminaries

2.1 Transformer

Transformers are DL models with strong capabilities in language modeling [17], computer vision [43], time series [126], code generation [54], and software security [30, 137]. A typical transformer [119] consists of a series of transformer layers stacked together. Each layer has two main components: *multi-head self-attention* and a *feedforward network (FFN)*. Fig. 2 illustrates a standard transformer architecture, with our targets highlighted in deep blue.

Multi-Head Self-Attention. Given input embeddings $X \in \mathbb{R}^{n \times d}$, the Query (Q), Key (K) and Value (V) vectors are computed as $Q = XW^Q$, $K = XW^K$, and $V = XW^V$, and then the attention scores are computed via $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$, where W^Q, W^K, W^V are three

learnable weight matrices and d_k denotes the dimensionality of Q and K . To better capture multiple types of relationships in the sequence at the same time, we typically split the attention mechanism into h heads, compute attention individually, and aggregate with a W^O matrix: $\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$, where $\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$.

Feedforward Network. The FFN applies two linear layers separated by a GELU activation: $\text{FFN}(X) = \text{GELU}(XW_1 + b_1) W_2 + b_2$. Residual connections and layer normalization follow each sub-layer. Our scheme focuses on encrypting the multi-head attention and FFN weights, which together account for the bulk of the parameters in a transformer model.

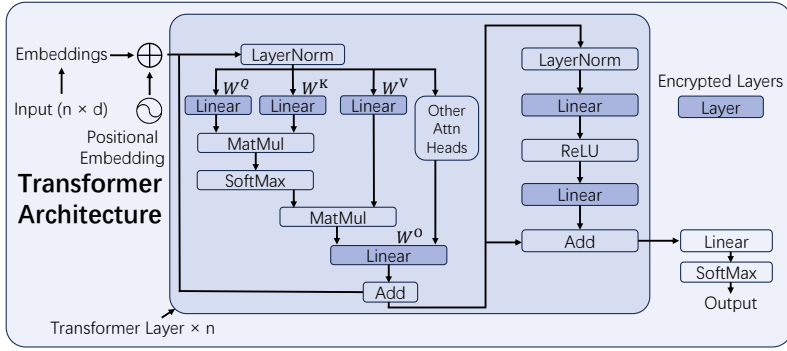


Fig. 2. Block diagram of a standard transformer layer. The components targeted by our scheme are denoted in deep blue (see later in Sec. 4.2), which account for most of the parameters in the transformer layer.

2.2 Arnold's Cat Map

Arnold's Cat Map (ACM) is a chaotic transposition technique originally introduced into image cryptography by Fridrich [35] as a more efficient alternative to general-purpose block ciphers for media data, exploiting the large capacity and high redundancy of images. Fridrich's original construction interleaves two phases per round: a *permutation* phase that shuffles pixel positions using a chaotic map and a *diffusion* phase that propagates information across the image to break local correlations. Since then, the ACM family has been adapted to medical image protection [19], voice communication encryption [31], and a series of generalizations enlarging the key space and dimensionality [114, 128, 140]. Lin et al. [68] were the first to repurpose ACM for neural-network weight obfuscation, applying it to CNN and RNN weights with a restricted two-parameter form. We build on this lineage but adapt it specifically to transformers: the attention projection matrices are square and naturally fit the ACM domain, while the FFN matrices are non-square and require a separate permutation primitive (see Sec. 4.2). To ensure robust protection that satisfies formal privacy guarantees against adversaries who possess a public reference model (Sec. 3), we further extend the standard ACM permutation with a value-diffusion step inspired by the original Fridrich construction.

Permutation Phase. The ACM scrambles the position of an element at coordinates (x_0, y_0) in an $S \times S$ matrix to a new position (x_n, y_n) over n iterations. We adopt the restricted form proposed by Lin et al. [68] for its efficiency. The coordinate transformation and its reversible decryption are

defined compactly as:

$$\underbrace{\begin{bmatrix} x_n \\ y_n \end{bmatrix}}_{\text{Encryption}} = A^n \underbrace{\begin{bmatrix} x_0 \\ y_0 \end{bmatrix}}_{\text{Decryption}}, \quad \underbrace{\begin{bmatrix} x_0 \\ y_0 \end{bmatrix}}_{\text{Decryption}} = A^{-n} \underbrace{\begin{bmatrix} x_n \\ y_n \end{bmatrix}}_{\text{Encryption}} \pmod{S} \quad (1)$$

where $A = \begin{bmatrix} 1 & p \\ q & pq+1 \end{bmatrix}$ is the transformation matrix with parameters $p, q \in \mathbb{Z}^+$. Since $\det(A) = 1$, the map is area-preserving and fully reversible.

Diffusion Phase (Optional). Pure permutation affects only the positional correlations but leaves the statistical distribution of values unchanged. To resist statistical analysis, we introduce an optional diffusion step. Let $v_{(x,y)}$ denote the raw l -bit representation of the parameter value after the permutation phase at position (x, y) . The diffusion operation XORs it with a pseudo-random l -bit mask $\mathcal{M}$ derived from the diffusion key:

$$v'_{(x,y)} = v_{(x,y)} \oplus \mathcal{M}_{(x,y)} \quad (2)$$

where $v'_{(x,y)}$ is the new raw bit string after diffusion. XOR preserves the storage width and is exactly reversible. The permutation and diffusion are implemented as separate streaming kernels in the order specified in Sec. 4.2.3. This ensures that even if the permutation key is leaked, the underlying weight values remain computationally hidden under the PRF and key-secrecy assumptions.

Our scheme encrypts the selected transformer layers rather than limiting protection to individual weights [125, 145]; this avoids the statistical vulnerability identified in [108].

2.3 Physical Unclonable Functions (PUF)

PUF is a hardware security primitive designed based on uncontrollable and unpredictable manufacturing process variations in circuit design [16, 106]. A PUF can be mathematically viewed as an irreversible probabilistic function that produces a random bit string (the response) given a challenge to the circuit:

$$\text{PUF} : \mathcal{C} \rightarrow \mathcal{R}, \quad ch \mapsto r. \quad (3)$$

Here, $ch \in \mathcal{C}$ and $r \in \mathcal{R}$. The exact physical nature of the challenge space $\mathcal{C}$ depends on the underlying PUF construction. For delay-based strong PUFs (e.g., Arbiter PUFs), the challenge is an explicit external bit-vector that configures internal signal routing paths, yielding an exponentially large challenge-response space. Conversely, for memory-based weak PUFs (e.g., SRAM PUFs), the challenge inherently corresponds to the specific limited physical memory addresses queried to extract the uninitialized power-on state. Because this enrolled memory footprint is permanently fixed, the primitive effectively operates with a single, monolithic challenge-response pair (CRP). The challenge-to-response mapping of a specific PUF is unique and irreproducible, just as no two people have identical fingerprints due to natural biological variations, making PUF a promising technique for generating device-specific keys and hardware identifiers.

In practice, raw PUF responses are inherently susceptible to environmental noise, and PUFs that expose many raw CRPs are vulnerable to modeling attacks. To address this, a technique called controlled PUF (CPUF) can be adopted so that PUFs can only be accessed via an algorithm that is physically bound to the PUF in an inseparable way [38]. In particular, the hardware designer can create the package so that its existence has a significant influence on the PUF measurement, and tampering intended to bypass the control logic perturbs the PUF response beyond its reconstruction tolerance, preventing correct key recovery. Meanwhile, the user or application engages with the PUF through a controlled and non-bypassable interface to reach the shared secret. This interface encapsulates the fuzzy extractor implementation, including error correction and privacy

amplification, to reconstruct a stable key statistically close to uniform. Provided that the interface cannot be bypassed, hiding the raw PUF readout and blocking arbitrary challenge queries prevents conventional CRP-harvesting modeling attacks through the exposed interface. In this work, it is assumed that the hardware provider incorporates such a control mechanism within the PUF design, ensuring that a physical adversary is unable to directly probe the PUF responses. Furthermore, the enrolled device response R_{dev} must retain sufficient conditional min-entropy given public helper data P . Following the leftover-hash lemma [28], a standard sufficient bound for extracting a κ -bit key whose distribution is within statistical distance ϵ of uniform is $H_{\infty}(R_{\text{dev}} | P) \geq \kappa + 2 \log_2(1/\epsilon)$. For $\kappa = 128$ and $\epsilon = 2^{-64}$, this conservative bound requires at least 256 bits of conditional min-entropy.

In our architecture, the CPUF control algorithm is used for both weak- and strong-PUF realizations and encapsulates response acquisition, fuzzy extraction, and access to the reconstructed root key [118]. For the SRAM weak PUF, it controls access to the enrolled startup region and prevents exposure of the raw response or root key through the user-facing interface; no external challenge is required. For a strong PUF, it additionally restricts challenge submission and prevents collection of raw challenge–response pairs [94].

Realizations on edge accelerators. ChaosFormer requires a device-unique, highly reliable key source with sufficient conditional min-entropy given public helper data, but is agnostic among PUF constructions that satisfy these requirements. However, it is useful to outline the realistic options for edge accelerators so that the hardware-bound claim is concrete. For example, Arbiter PUFs [105] exploit the propagation-delay variation between identically routed signal paths and are widely deployed on FPGAs because they reuse standard logic resources. SRAM PUFs [45] read out the random power-on state of uninitialized SRAM cells and are attractive for microcontroller-class edge devices, where SRAM is already on-chip. An SRAM region used as a PUF must be read in its uninitialized power-on state; implementations can obtain this state through device startup or dedicated local power control [52]. DRAM-based variants [58, 112] reuse main memory either through capacitor decay or through deliberately violated row-activation timing, which makes them well-suited to accelerators that expose DRAM but cannot afford additional silicon. Practical edge deployments can use commercially hardened PUF IP integrated into the system-on-chip, such as Intrinsic ID’s SRAM-PUF technology [52]. Its published qualification summary reports an average SRAM-PUF response-noise level below 15% across tests spanning -55 to 150°C , $\pm 20\%$ voltage variation, humidity, and electromagnetic interference. The same report states that its error-correction and privacy-amplification implementation achieves a typical key-reconstruction error rate below 10^{-12} and incorporates an anti-aging technique for a reported 25-year lifetime [52, 76]. For ChaosFormer, the enrolled source must satisfy the conditional-min-entropy bound stated above, reproduce the key throughout the deployment’s qualified operating envelope, and fail closed when decoding is unsuccessful. The device reconstructs K_{master} once during secure initialization and directly derives a device–owner–model–version–scheme key during authorized model loading; these secrets are absent from the encrypted artifact in transit or at rest.

3 Threat Model

We consider a deployment scenario where transformer models are executed on edge devices. We assume the hardware root-of-trust (e.g., PUF) functions correctly and is tamper-resistant against physical extraction, ensuring that the PUF readout and reconstructed keys are unavailable through the user-facing interface or device-accessible non-volatile storage; the vendor’s separately protected enrollment record is an explicit trust anchor. We exclude invasive hardware attacks (e.g., decapping) and direct side-channel attacks on the PUF circuitry itself from our threat model, as these require specialized equipment and are orthogonal to the proposed algorithmic protection. Furthermore, we assume the PUF is strictly encapsulated within a CPUF architecture whose non-bypassable

interface performs fuzzy extraction and prevents exposure of raw responses and reconstructed root keys. However, we assume a powerful adversary with *white-box access* to the encrypted model file. Specifically, the attacker possesses:

- **Model Knowledge:** Full access to the encrypted parameters and model architecture.
- **Data:** Access to a subset of the original training data and full downstream datasets.
- **Computational Resources:** Sufficient power to attempt model recovery via retraining, fine-tuning, or brute-force key search.
- **Timing Side-Channel Information:** The ability to observe inference latency, potentially revealing which layers are encrypted via timing analysis (addressed in Sec. 5.2.2).

Table 1 summarizes the attacker capabilities admitted under our threat model. Indirect black-box model extraction [84, 85, 116], which queries a deployed model to train a substitute, is outside our analysis because reconstructing a transformer through queries can require a prohibitive number of samples [34, 63]. Our scheme preserves the original model’s accuracy on the authorized device, so a query-based extractor sees no accuracy degradation that it could exploit. The standard threat model captures proprietary, train-from-scratch models for which the attacker lacks an aligned plaintext checkpoint; this is the setting addressed by the base ChaosFormer scheme, which provides robust IP protection while optimizing for latency-sensitive, efficiency-first edge deployments. Conversely, the extended threat model accounts for the deployment of fine-tuned derivatives based on open-weight foundation models (e.g., LLaMA or OPT). In this setting, a powerful adversary possesses the aligned public pretrained checkpoint, enabling potential structural alignment attacks.

Table 1. Standard and extended threat models. A checkmark (✓) indicates that the attacker has access to the corresponding information or resource; a cross (×) indicates that access is restricted under our threat model.

Attacker Information	Standard	Extended
Model architecture	✓	✓
Offline storage access (encrypted weights)	✓	✓
Partial training data / downstream datasets	✓	✓
Public pre-trained reference model	×	✓
Inference latency / timing observations	✓	✓
Privileged runtime/debug access to plaintext	×	×
PUF root and descendant keys	×	×
PUF readout interface	×	×
Invasive hardware probing of PUF circuitry	×	×

Scheme applicability: ChaosFormer targets the Standard setting. When the Extended setting admits an aligned public checkpoint, the diffusion-augmented ChaosFormer-S is required; ChaosFormer-S also applies under the Standard setting.

Security Guarantees and Runtime Trust Boundary. Our evaluated guarantee concerns the encrypted artifact: a party that copies the checkpoint from storage, distribution, or an unauthorized device cannot reconstruct functional weights without the device-bound key. Authorized inference necessarily reconstructs each protected layer before its operator executes. We therefore assume a production runtime in which the adversary cannot attach a debugger, load a kernel memory-acquisition module, inspect accelerator global memory, or modify the signed inference binary or driver. This trust boundary relies on standard OS-level privilege separation (e.g., kernel-enforced sandboxes and the default disabling of diagnostic interfaces in locked production devices [4, 5]) rather than the obfuscation primitive itself. By contrast, successful memory-recovery attacks against

deep learning models typically presuppose forensic-level capabilities, such as acquiring complete CPU/GPU memory snapshots via kernel modules or hardware debuggers [86]. Consequently, a fully compromised OS (e.g., a rooted device), a malicious kernel or driver, or an exposed hardware diagnostic port inherently violates our runtime assumption. Layer-wise decryption and immediate erasure reduce the simultaneously exposed plaintext and its residence time, but do not prevent such a privileged adversary from accumulating layers over repeated runs. Deployments requiring that stronger property must add a hardware-enforced protected runtime, secure accelerator, or fused decrypt–compute path with attestation and debug/DMA restrictions.

Extended Threat Model. As Transformers are typically fine-tuned from large, publicly available pre-trained models (e.g., BERT, ViT available on Hugging Face), a sophisticated adversary might possess not only the encrypted weights but also the original pre-trained base model. We make this extended assumption explicit because it is realistic—many transformer models in deployment are initialized from a small set of public checkpoints—and because it enables a concrete structural alignment attack. The base ChaosFormer permutation, on its own, preserves the empirical distribution of weights in each layer: the histogram, mean, variance, and higher-order moments are all invariant under permutation. This opens a concrete *alignment attack* against pure-permutation schemes:

- (1) For each layer i , the attacker computes column-wise summary statistics (mean, variance, ℓ_2 norm) on both the public checkpoint W_i^{pub} and the encrypted release W_i^{enc} .
- (2) Because permutation Π rearranges columns but does not change their internal values, the multiset of column-statistic vectors of W_i^{enc} is identical to that of W_i^{pub} up to fine-tuning drift.
- (3) The attacker then solves a linear assignment problem (e.g., the Hungarian algorithm) to match columns of W_i^{enc} to columns of W_i^{pub} , recovering the permutation up to cosmetic ambiguity.

This style of attack has been demonstrated against TEE-based permutation defenses [123] and is exactly why pure permutation is insufficient against an adversary with the public checkpoint. ChaosFormer-S’s diffusion step destroys the alignment signal by masking each raw weight encoding with a PRF keystream derived from the final model–version–scheme key $K_{m,v,s}$, which Sec. 4.1 derives from the PUF-backed master key and authenticated deployment context. Under the PRF and key-secrecy assumptions, the resulting encodings are computationally indistinguishable from uniform over $\{0, 1\}^l$, so step (2) above no longer holds. Sec. 4.2.1 formalizes this guarantee as Single-Point Indistinguishability.

Concrete Deployment Scenarios. Our threat model (Table 1) captures three classes of deployment routinely. First, edge devices that receive periodic firmware or model updates (e.g., wearables, automotive ECUs, home IoT hubs) typically pull updates through an intermediary smartphone or gateway. A compromise of that intermediary, or a man-in-the-middle on the update channel, exposes the model file in transit [26, 98, 110]; Second, centralized server deployments are routinely targeted through insider attacks [51] and lateral phishing [27, 44], both of which give the adversary read access to model checkpoints without ever requiring memory-level compromise during execution. Third, supply-chain vulnerabilities arise when models pass through several organizations during development, hardware integration, and deployment, creating multiple checkpoints at which an at-rest model file can be exfiltrated. In each scenario, the attacker possesses exactly the assets that Table 1 grants them: the encrypted model on disk, the model architecture (often public), and possibly some training data—but not the noisy device response R'_{dev} acquired during key reconstruction, the stable PUF-backed root key K_{master} , or any key derived from that root. This is the regime in which ChaosFormer is designed to make the stolen model worthless.

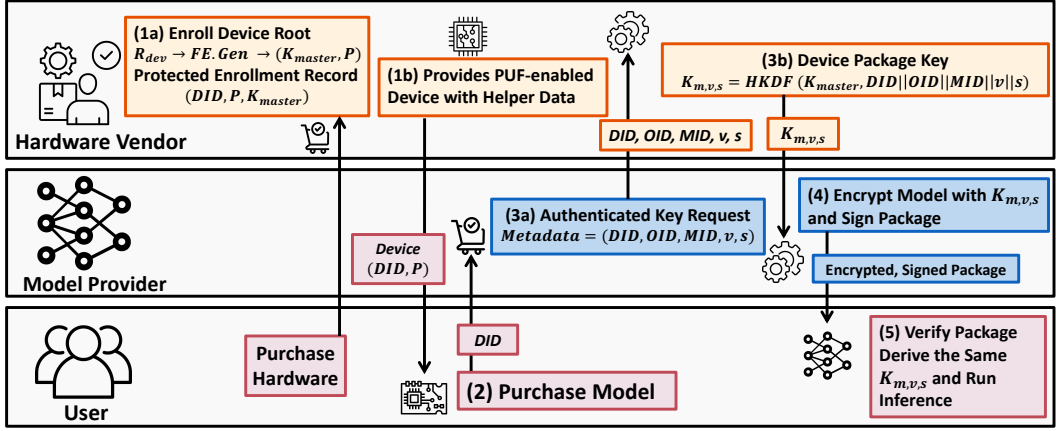


Fig. 3. The protocol of the interaction between the Hardware Vendor, Model Provider, and User. The protocol is used to ensure secure model deployment.

4 Proposed Approach

This section presents our proposed ChaosFormer. The overall framework can be divided into three stages. In the first stage, the model provider trains the model using its training expertise, computing resources, and proprietary datasets. In the second stage, users or companies request to purchase and deploy the protected model. The provider interacts with them using a specific protocol, detailed in Sec. 4.1, encrypts the model weights based on the identity of the purchaser and the designated device characteristics derived from its PUF. After the purchase, the model can be deployed only on the specific machine. Eavesdropping or stealing weights from storage no longer makes them usable.

4.1 Secure Deployment Protocol

Securely deploying a transformer model on PUF-equipped hardware involves a coordinated protocol among three main entities: Hardware Vendor, Model Provider, and User. Fig. 3 depicts the protocol, with numbers denoting the different stages and letters denoting actions taken by different parties in a stage. Table 2 summarizes the meaning, visibility, and lifetime of the protocol variables. The protocol progresses through the following stages:

(1) Hardware Acquisition and Registration: (a) The Hardware Vendor is assumed to be trusted and is responsible for enrolling the device PUF. The vendor temporarily accesses the chip's manufacturing test interface to extract the PUF baseline R_{dev} via a secure internal mechanism (e.g., reading power-on memory states or executing a vendor-configured root challenge). Using this baseline, the vendor generates the public helper data P and computes the stable, device-specific master key K_{master} via the fuzzy-extractor procedure $(K_{master}, P) = FE.Gen(R_{dev})$. The vendor stores the association among the device identifier DID , P , and K_{master} in a secure offline database. As a deployment requirement, the vendor then irreversibly disables the unrestricted raw-readout test interface (e.g., using one-time programmable fuses), leaving only the CPUF's encapsulated internal key-derivation paths active. The DID is embedded in device read-only memory, while the public helper data P is kept in authenticated, rollback-protected device storage so that authenticated re-enrollment can replace it. (b) The User then acquires the PUF-enabled hardware (e.g., an FPGA or domain-specific accelerator) from the Hardware Vendor.

Table 2. Notation and lifecycle of the key-derivation hierarchy in the secure deployment protocol.

Symbol	Meaning	Visibility and lifetime
$R_{\text{dev}}, R'_{\text{dev}}$	Enrolled and noisy runtime PUF responses	Raw values accessible only within the CPUF-controlled acquisition and fuzzy-extraction path; R'_{dev} exists transiently during secure initialization
P	Fuzzy-extractor helper data	Public; persistent in authenticated, rollback-protected device storage and the enrollment record; replaced upon re-enrollment
DID	Device identifier	Public, persistent, and signed as package metadata; binds the hierarchy to the enrolled device record
OID	Human- or organization-readable owner identifier	Public, persistent, and signed as package metadata; not relied upon for entropy
MID, v, s	Model, version, and scheme identifiers	Signed package metadata; $v = 0$ is the initial release and $s \in \{\text{base, secure}\}$
K_{master}	Device root key reconstructed by the fuzzy extractor	Long-lived; retained in the vendor's protected enrollment record and reconstructed within the device's CPUF-controlled key path; never exposed through the user-facing interface
$K_{m,v,s}$	Device-owner-model-version-scheme encryption key	Sent to the provider over the secure channel and discarded after encryption; independently rederived by the device

(2) Model Purchase Request: The User, intending to deploy a specific DL model, contacts the Model Provider. In the request, the User provides the device identifier DID .

(3) Context-Specific Key Material Retrieval: (a) Once the user purchase request is received, the Model Provider sends DID , the model ownership identifier OID , the model identifier MID , the model version v , and the scheme identifier s to the Hardware Vendor. The initial model release uses $v = 0$, and $s \in \{\text{base, secure}\}$ distinguishes ChaosFormer from ChaosFormer-S. (b) After authenticating the request, the vendor looks up the K_{master} associated with DID and directly derives the context-specific key on its trusted server:

$$K_{m,v,s} = \text{HKDF}(K_{\text{master}}, DID \parallel OID \parallel MID \parallel v \parallel s). \quad (4)$$

We denote $\text{HKDF}(k, \text{info})$ as the HMAC-based Key Derivation Function (HKDF) extract-and-expand procedure instantiated with HMAC-SHA-256, utilizing k as the input key material and the displayed concatenation as the context string. The displayed fields use an unambiguous length-delimited encoding. The public OID is binding context rather than key entropy: the unpredictability of $K_{m,v,s}$ follows from the secret K_{master} . The vendor shares only $K_{m,v,s}$ with the Model Provider through an established secure channel.

(4) Model Encryption and Configuration: Upon receiving $K_{m,v,s}$, the Model Provider treats it strictly as an ephemeral secret and uses it to obfuscate the corresponding model checkpoint in a secure environment. To ensure authenticity and prevent metadata spoofing, the provider computes a digital signature over the obfuscated package firmly bound to its metadata (DID , OID , MID , v , and s) using a standard digital signature algorithm (e.g., ECDSA) under its long-term private key, then securely discards $K_{m,v,s}$ immediately after signing. The Model Provider delivers the signed package to the User, who can deploy it on the designated PUF-enabled hardware.

(5) PUF-Driven In-Situ Decryption: The secure runtime at the device side first verifies the model provider's signature using a provisioned provider public key. Upon success, the CPUF triggers the underlying physical primitive to obtain a noisy raw response R'_{dev} and internally reconstructs the stable root key $K_{\text{master}} = \text{FE.Rep}(R'_{\text{dev}}, P)$. The CPUF architecture then routes K_{master} and the authenticated DID , OID , MID , v , and s metadata into its integrated HKDF to compute the version-specific key $K_{m,v,s}$. After validating the package version against the rollback-protected version state, the CPUF feeds $K_{m,v,s}$ to de-obfuscate the model, typically layer by layer as needed for computation. Throughout this process, the raw PUF readout and derived keys (K_{master} and $K_{m,v,s}$) remain inside the hardware cryptographic boundary and are never returned through the user-facing interface.

Model updates, versioning, and re-enrollment. Authorized model updates do not require the model provider to store long-term model obfuscation keys ($K_{m,v,s}$), nor do they consume new CRPs. Instead, the core mechanism relies on cryptographic domain separation. The CPUF architecture internally reconstructs the single, stable device root K_{master} . To securely accommodate continuous updates, the hardware vendor dynamically derives an ephemeral, context-specific update key $K_{m,v,s}$ using an HKDF that binds the root key to the authenticated metadata (DID , OID , MID , v , and s). Because key unpredictability follows from the hardware-bound K_{master} , this derivation provides cryptographic separation across authenticated contexts and applies to PUF realizations that satisfy the stated entropy, reconstruction, and controlled-interface requirements without exposing the raw physical interface.

The update workflow is strictly forward-moving. Upon authenticating a request for a new version v , the vendor transmits the derived $K_{m,v,s}$ to the provider, who encrypts the new complete model checkpoint, signs the package, and immediately discards the key. On the edge device, the secure runtime verifies the provider's signature, reconstructs K_{master} , and checks a monotonic version counter maintained in rollback-protected trusted state. For a newly installed update, the incoming version v must be strictly greater than the stored counter; after the authenticated installation succeeds, the runtime atomically advances the counter to v . Subsequent boots may reload the accepted active version, whereas any package with a lower version is rejected. This rule prevents rollback to an older, vulnerable ciphertext without preventing normal reuse of the currently authorized checkpoint. The protocol also defines fail-closed behavior under physical device aging: if PUF noise exceeds the fuzzy extractor's correction budget, the device fails closed. Restoring functionality requires authenticated re-enrollment to replace the vendor's enrollment record, securely provision new public helper data P to the device, establish a new K_{master} , and issue a newly encrypted package; ciphertext bound to the deprecated enrollment is not reused.

This protocol ensures that a stolen package remains non-functional on another device: the other device reconstructs a different K_{master} and therefore cannot reproduce the required $K_{m,v,s}$.

4.2 PUF-based Single-layer Obfuscation

Fig. 4 depicts the per-layer key expansion after the device's CPUF key path has derived the final model-version-scheme key $K_{m,v,s}$ from its PUF root K_{master} as specified in Sec. 4.1. For each protected transformer layer i , an HKDF keyed with $K_{m,v,s}$ derives the layer key K_i , which is then split into the attention key K_i^{attn} and the FFN key K_i^{FFN} :

$$K_i = \text{HKDF}(K_{m,v,s}, i), \quad (5)$$

$$K_i^{\text{attn}} \parallel K_i^{\text{FFN}} = K_i. \quad (6)$$

Here, $\parallel$ denotes concatenation. Each K_i supplies independent material for obfuscating the self-attention weights ($W_i^Q, W_i^K, W_i^V, W_i^O$) and FFN weights (W_i^1, W_i^2).

ChaosFormer-S further requires a cryptographically stronger diffusion key stream $\mathcal{M}$. To achieve this, it utilizes a 20-round IETF ChaCha20 construction [12] as a secure Pseudo-Random Function (PRF). To securely seed this PRF, a domain-separated HKDF derives a 256-bit key and 96-bit nonce for each protected tensor from the final model-version-scheme key $K_{m,v,s}$ and the layer/tensor identity. Formally, for each protected tensor t ,

$$K_t^{\text{diff}} \parallel N_t = \text{HKDF}(K_{m,v,s}, i \parallel \text{id}(t)) \quad (7)$$

where i is the layer index and $\text{id}(t)$ is a length-delimited encoding of the parameter path of the within-layer tensor t in the model checkpoint (e.g., `attention.q_proj.weight`), so that $i \parallel \text{id}(t)$ uniquely identifies every protected tensor and is reconstructed identically during encryption and decryption. The first 256 bits of the HKDF output form the ChaCha20 key K_t^{diff} and the remaining 96 bits form the nonce N_t ; the PRF then consumes both the key and the nonce to generate the per-tensor keystream $\mathcal{M}_t$, which is truncated to match the exact size of the target tensor and subsequently consumed by the diffusion step Ψ .

$$\mathcal{M}_t = \text{ChaCha20}(K_t^{\text{diff}}, N_t) \quad (8)$$

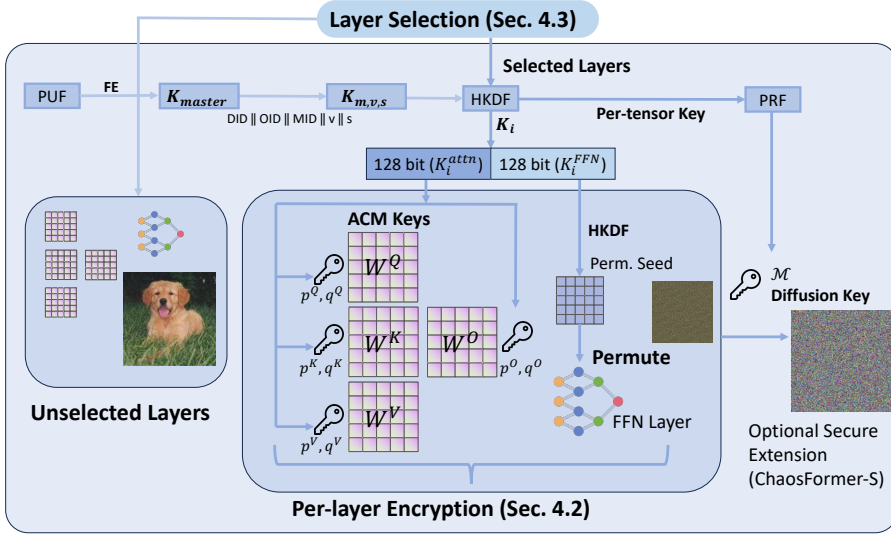


Fig. 4. Per-layer key expansion and weight obfuscation on the device. The device reconstructs its PUF root $K_{\text{master}} = \text{FE.Rep}(R'_{\text{dev}}, P)$ and re-derives the model-version-scheme key $K_{m,v,s}$ via HKDF as specified in Sec. 4.1; here $K_{m,v,s}$ expands into per-layer permutation material and, for ChaosFormer-S, per-tensor diffusion material.

4.2.1 Format-Preserving Encryption (FPE). To ensure that the encrypted model can be deployed on edge devices without altering data types, storage, or framework requirements, we formulate our approach as Format-Preserving Encryption (FPE) [11], where encryption does not alter the format of the data (as a runnable model). Although Transformer weights are typically floating-point numbers (e.g., FP32, BF16), digital systems represent them as discrete bit strings. For an l -bit weight format, we therefore define the per-weight message space as $\mathcal{D} = \{0, 1\}^l$. Permutation rearranges these fixed-width encodings, while diffusion XORs them with an equally long keystream. Every

ciphertext remains a valid l -bit storage value without numerical conversion or expansion. Our framework offers two variants targeting different threat models: ChaosFormer and ChaosFormer-S. **Why format preservation matters in this setting:** The FPE framing is not merely a mathematical convenience; it reflects three concrete deployment constraints. First, the encrypted file must continue to load through standard frameworks (e.g., HuggingFace Transformers' `from_pretrained`), which validate tensor shapes and dtypes when reading a checkpoint — a generic block cipher would lengthen each weight to a multiple of 128 bits and break this validation. Second, mixed-precision and quantization-aware deployments rely on the bit width of each weight matching exactly what the model graph expects (e.g., 8-bit integer kernels assume 8-bit operands), so any encryption that changes the bit width forces the deployment pipeline to be re-tooled per device. Third, the on-device key release is layer-by-layer (Sec. 4.1); a non-format-preserving scheme would require an in-place padding/unpadding pass at every layer, whereas an FPE allows the decrypted layer to be consumed directly by the next operator without any reshaping. This is also what distinguishes our approach from generic at-rest encryption (e.g., AES-CBC over the whole checkpoint): FPE makes per-layer in-situ decryption a $O(\text{layer size})$ operation that returns a tensor of the original shape and dtype, ready for matmul.

DEFINITION 1 (FORMAT-PRESERVING ENCRYPTION [11]). A *Format-Preserving Encryption (FPE)* scheme is a function $E : \mathcal{K} \times \mathcal{F} \times \mathcal{T} \times \mathcal{X} \rightarrow \mathcal{X} \cup \{\perp\}$, where $\mathcal{K}$ is the key space, $\mathcal{F}$ is the format space, $\mathcal{T}$ is the tweak space, and $\mathcal{X}$ is the domain. For a fixed key K , format F , and tweak T , the function $E_K^{F,T}(\cdot)$ acts as a permutation on the slice $X_F \subset \mathcal{X}$, ensuring that for any input $X \in X_F$, the output $Y = E_K^{F,T}(X)$ remains within X_F (i.e., preserves the format).

1) ChaosFormer (Base Scheme): Designed for the standard threat model, the proposed ChaosFormer relies on a cryptographic permutation to scramble weight positions, thereby disabling model capabilities. For attention weights, we utilize Arnold's Cat Map (ACM) as the permutation function Π_{ACM} . For non-square matrices (FFN weights), we utilize the Knuth Shuffle as the permutation function Π_{Knuth} .

$$\begin{aligned} W_i^{j'} &= \Pi_{\text{ACM}}(W_i^j, (p_i^j, q_i^j)) & \text{for } j \in \{Q, K, V, O\} \\ W_i^{f'} &= \Pi_{\text{Knuth}}(W_i^f, \text{HKDF}(K_i^{\text{FFN}}, f)) & \text{for } f \in \{1, 2\} \end{aligned} \quad (9)$$

Here, i denotes the i -th layer, and the ACM parameters (p_i^j, q_i^j) are obtained by partitioning K_i^{attn} into non-overlapping blocks.

2) ChaosFormer-S (Secure Extension): To resist statistical attacks from adversaries possessing a public reference model (as discussed in Sec. 3), ChaosFormer-S extends the base scheme with a diffusion step Ψ on the permuted weights $\Pi(W)$. Let $\mathcal{M}$ be a keystream generated by a cryptographically secure Pseudo-Random Function (PRF) from domain-separated per-tensor key material derived from $K_{m,v,s}$ (cf. eq. (8)). The ciphertext is computed as:

$$W_{\text{enc}} = \Psi(\Pi(W), \mathcal{M}) \equiv \Pi(W) \oplus \mathcal{M} \quad (10)$$

This bitwise masking makes the encrypted representation computationally indistinguishable from a uniformly random bit string under the PRF assumption.

Visual demonstration. To visually validate the efficacy, we present a demonstration using sample images. As illustrated in Fig. 5, the ChaosFormer effectively destroys the spatial correlation of the original images through ACM permutation, rendering the visual content completely unrecognizable. However, as a pure permutation, it preserves the original pixel value histogram. In contrast, the ChaosFormer-S incorporates the diffusion step Ψ . The resulting output is visually identical to white noise, with the underlying pixel distribution fully masked. This visualization illustrates

the distribution-masking effect; the computational indistinguishability claim follows from the PRF-based argument above.

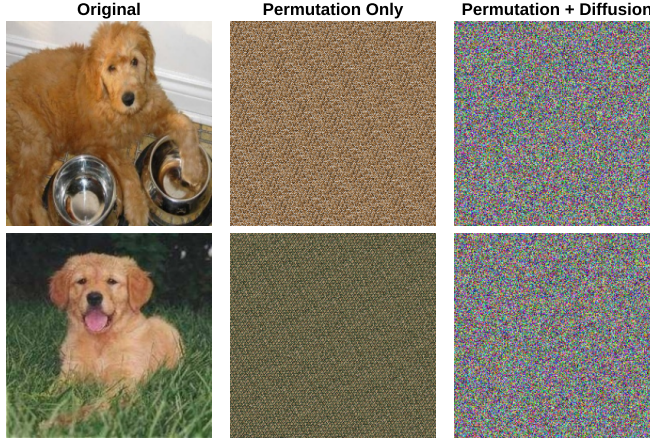


Fig. 5. Visual demonstration of the encryption schemes.

Formal Security Guarantee. ChaosFormer-S is designed to satisfy Single-Point Indistinguishability (SPI), a rigorous security notion introduced by Bellare et al. [11] that implies an adversary cannot distinguish the encryption of a message from a random string, *even if they choose the message themselves*. For example, they could purchase a model that is known to build upon a known pre-trained model.

DEFINITION 2 (SINGLE-POINT INDISTINGUISHABILITY (SPI) [11]). Let $\mathcal{A}$ be an adversary and $\mathcal{E}$ be an FPE scheme. Consider an experiment where a random bit $b \in \{0, 1\}$ is chosen. The adversary queries a single message X with format N and tweak T . If $b = 1$, the adversary receives the real ciphertext $Y = E_K^{N,T}(X)$; if $b = 0$, they receive a random string $Y \xleftarrow{\$} X_N$. The scheme is SPI-secure if the adversary's advantage, defined as $\text{Adv}_{\mathcal{E}}^{\text{spi}}(\mathcal{A}) = |\Pr[\mathcal{A} \rightarrow 1 | b = 1] - \Pr[\mathcal{A} \rightarrow 1 | b = 0]|$, is negligible.

THEOREM 1. If the keystream $\mathcal{M}$ is indistinguishable from uniform random noise (i.e., the PRF is secure), then ChaosFormer-S achieves SPI security.

PROOF. This theorem is formally proved using the Sequence-of-Games approach [104] by defining two games.

Game 0 (Real Scheme): The adversary interacts with the real obfuscation oracle where the mask $\mathcal{M}$ is generated by a PRF.

Game 1 (Ideal Scheme): The adversary interacts with a modified oracle where the mask $\mathcal{M}$ is replaced by a truly random string $\mathcal{M}^* \xleftarrow{\$} \{0, 1\}^{lN}$, where N denotes the number of l -bit weights. An encoded N -parameter tensor is therefore an element of $\mathcal{D}^N = \{0, 1\}^{lN}$.

Let the adversary $\mathcal{A}$ choose a fixed encoded message $W \in \{0, 1\}^{lN}$. Let $W' = \Pi(W)$ be the permuted weight tensor. Since Π is deterministic, W' is fixed. Under the ideal model assumption, the real ciphertext is the random variable $C_{\text{real}} = W' \oplus \mathcal{M}^*$. Since $\mathcal{M}^*$ is uniform over $\{0, 1\}^{lN}$, XOR with the fixed string W' is a bijection. Thus, for every $y \in \{0, 1\}^{lN}$,

$$\Pr[C_{\text{real}} = y] = \Pr[\mathcal{M}^* = y \oplus W'] = 2^{-lN}.$$

Consequently, $C_{real} \sim U(\{0, 1\}^{IN})$. This distribution is identical to that of the random string $C_{rand} \xleftarrow{\$} \{0, 1\}^{IN}$ used in the SPI experiment ($b = 0$). Therefore, the adversary's advantage in *Game 1* is:

$$Adv_{\mathcal{E}}^{spi}(\mathcal{A} - \text{Game 1}) = |\Pr[\mathcal{A}(C_{real}) \rightarrow 1] - \Pr[\mathcal{A}(C_{rand}) \rightarrow 1]| = 0.$$

The only difference between *Game 0* and *Game 1* is the source of the mask $\mathcal{M}$. If an adversary $\mathcal{A}$ can distinguish the output of ChaosFormer-S from random noise with a non-negligible advantage, they can effectively distinguish the PRF keystream $\mathcal{M}$ from a truly random string $\mathcal{M}^*$. This contradicts the assumption that the underlying PRF is secure. Therefore, the advantage is bounded by the security of the PRF:

$$Adv_{\mathcal{E}}^{spi}(\mathcal{A}) = Adv_{\mathcal{PRT}}(\mathcal{A}) + Adv_{\mathcal{E}}^{spi}(\mathcal{A} - \text{Game 1}) \approx \text{negligible}$$

□

While ChaosFormer-S provides stronger security guarantees, ChaosFormer offers lower latency by omitting the diffusion step. In our experiments, we evaluate the trade-offs between these two variants.

4.2.2 More Details on ACM. Now we provide a formal proof that ACM, defined by the transformation matrix $A = \begin{bmatrix} 1 & p \\ q & pq + 1 \end{bmatrix}$ for $p, q \in \mathbb{Z}^+$, constitutes a **chaotic** system, which is known as the “avalanche effect” in cryptographic literature.

DEFINITION 3 (CHAOTIC SYSTEM [140]). A discrete dynamical system is **chaotic** if:

- (1) Its dynamics are confined to a **globally bounded phase space**.
- (2) It possesses at least one **positive Lyapunov exponent**, indicating sensitivity to initial conditions (avalanche effect).

DEFINITION 4 (LYAPUNOV EXPONENT). For a linear map with a constant Jacobian matrix A , the Lyapunov exponents λ_k are defined as $\lambda_k = \ln(|\mu_k|)$, where μ_k are the eigenvalues of A .

THEOREM 2. The Arnold's Cat Map is a chaotic system for any positive integers p, q .

PROOF. We demonstrate that the system satisfies both conditions of Definition 1.

1. Bounded Phase Space: The map operates on the torus $[0, S) \times [0, S)$ via the modulo S operation. Since every output state (x_{n+1}, y_{n+1}) is mapped back into this finite region, the phase space is globally bounded.

2. Positive Lyapunov Exponent: We compute the eigenvalues μ of the transformation matrix A . The characteristic equation $\det(A - \mu I) = 0$ expands to:

$$(1 - \mu)(pq + 1 - \mu) - pq = \mu^2 - (2 + pq)\mu + 1 = 0 \quad (11)$$

Solving for μ yields two eigenvalues:

$$\mu_{1,2} = \frac{(2 + pq) \pm \sqrt{(2 + pq)^2 - 4}}{2} \quad (12)$$

Consider the larger eigenvalue μ_1 . Since $p, q \geq 1$, we have $pq \geq 1$, implying that the discriminant is positive. It follows that:

$$\mu_1 = \frac{2 + pq + \sqrt{4pq + (pq)^2}}{2} > \frac{2 + pq}{2} \geq 1.5, \quad \lambda_1 = \ln(|\mu_1|) > \ln(1) = 0 \quad (13)$$

The existence of a positive Lyapunov exponent confirms the system exhibits the requisite sensitivity to initial conditions. Thus, the ACM is chaotic. □

4.2.3 Performance Optimizations. To ensure ChaosFormer and ChaosFormer-S are practical for real-time edge deployment and to mitigate leakage of the secret ACM iteration count through execution timing, we employ several critical optimizations in our implementation.

Mitigating ACM Iteration-Count Timing Leakage. A naive ACM implementation applies the coordinate transformation in eq. (1) iteratively n times, making its execution schedule depend directly on the secret iteration count. We mitigate this timing channel by removing two sources of n -dependent computation: repeated tensor mappings and a variable matrix-product count.

Secret-independent GPU work schedule (single-pass kernel). We derive the cumulative transformation matrix $A_{final} = A^n \pmod{W}$ and apply the corresponding coordinate mapping once, replacing n successive tensor mappings with a single inverse-mapping gather. The kernel grid, number of output writes, tensor volume, and number of launches are consequently fixed by the weight shape rather than by n . This cumulative mapping removes the prominent n -dependent tensor-work signal of the naive implementation. In Fig. 8, the protected GPU inference overhead remains approximately $1.10\times$ across $n = \{3, 5, 8, 12, 16, 32\}$, providing empirical evidence that the tested implementation has no observable iteration-count timing trend at our end-to-end measurement granularity.

Fixed-schedule host matrix exponentiation. Standard binary exponentiation computes A_{final} in $O(\log n)$ matrix products, but the exact product count still depends on the bits of n . We therefore implement a fixed schedule for the supported range $n \in [3, 34]$: six rounds always compute both the square and candidate multiply, resulting in 12 modular 2×2 products, and select the result by arithmetic masking. Across three randomized sweeps, the variable schedule has a $2.169\text{--}2.174\times$ max/min timing spread, whereas the fixed schedule reduces this spread to $1.003\text{--}1.006\times$. The run-median fixed/original GPU encrypt-decrypt ratio is $0.9992\text{--}1.0001\times$, with all per- n ratios lying within $0.9953\text{--}1.0066\times$. Thus, the fixed schedule removes the n -dependent matrix-product count without measurable end-to-end cost. Because the exponentiation operates only on 2×2 matrices, its cost is computationally negligible relative to the memory-bound tensor permutation.

Together, the cumulative tensor mapping and fixed-schedule exponentiation remove the two algorithmic sources of n -dependent work and empirically mitigate iteration-count timing leakage at the measurement granularity used in our evaluation. Permutation-specific memory accesses and runtime behavior can still vary, so this result is not a universal machine-level constant-time guarantee. It specifically addresses the iteration-count timing channel; the remaining excluded side channels are detailed in Sec. 6.4.

Inverse-Mapping for Coalesced Memory Access. In the GPU implementation, directly mapping input coordinates (x, y) to permuted destinations (x', y') (scatter) results in non-contiguous memory writes, which severely degrades memory bandwidth utilization. Instead, we adopt an *inverse-mapping gather* strategy using Triton kernels. We assign GPU threads to consecutive *output* coordinates and calculate their corresponding *source* coordinates using the inverse transformation matrix A_{final}^{-1} . This ensures that all write operations to the output tensor are contiguous and fully coalesced, maximizing memory throughput.

On-the-Fly Cryptographic Keystream. The Triton kernel [113] computes independent 512-bit ChaCha20 blocks across vector lanes, XORs them directly into the tensor, and discards the state; it therefore allocates no tensor-sized keystream. The permutation and diffusion remain separate streaming kernels so that encryption applies diffusion after permutation and decryption removes it before the inverse permutation. This explicit ordering preserves exact bitwise reversibility and coalesced writes in the inverse-mapping gather.

Layer-wise dtype handling and key setup. Diffusion operates on the raw bit pattern of each contiguous tensor (Sec. 4.2.1); FP32, BF16, FP16, and INT8 storage is reinterpreted as 32-bit words without numerical conversion. The software prototype pre-derives the small per-tensor key and

nonce records during cipher setup, before protected inference, and passes them as scalar kernel arguments. Thus, recurring inference pays for ChaCha20 generation and one streaming XOR pass but not host-side KDF execution or a VRAM-resident key tensor. At ViT-B batch size 64, replacing the prototype generator with ChaCha20 changes protected-pipeline latency by $1.014\times$ at $K = 6$ and $1.021\times$ when all 12 layers are protected. Together with binary exponentiation and inverse-mapping gather, this yields the overhead profile evaluated in Sec. 5.3.

4.3 Full Model Obfuscation

In our base threat model, where the adversary lacks access to a reference public model (e.g., a pre-trained base model from a public repository), extensive encryption of every parameter is not strictly necessary to disable the model's functionality. We evaluate three layer-selection strategies: 1) greedy Top- K , which grows the protected set one layer at a time and, at each round, profiles every remaining layer when added to the current set before selecting the candidate that causes the largest testing-accuracy drop; 2) Random- K , which randomly draws K layers; and 3) Last- K , which encrypts the last K transformer layers in the model.

Note on layer-selection design. Our preliminary work [48] introduced the iterative greedy layer-selection algorithm retained here as Top- K . We compare it with Random- K and Last- K to separate the effect of the selection rule from the encryption budget. Our controlled study in Sec. 5.5 shows that Random- K is heterogeneous: it can sample Top- K -level robustness without running the greedy profiling search, but is not uniformly stronger. The greedy algorithm also has a subtle weakness once the extended threat model is admitted—an attacker who knows that the encrypted layers follow the deterministic Top- K selection path can use public-checkpoint salience to prioritize targeted repair. Random- K removes this deterministic selection cue and adds uncertainty against targeted repair.

As shown in Fig. 6, our complete protection workflow begins with a well-trained transformer model (e.g., ViT-B). The selected (or full) layers are then encrypted with $K_{m,v,s}$ obtained through the hardware-provider protocol in Sec. 4.1. This approach allows the same plaintext model to be issued as separately encrypted packages: changing any authenticated device, owner, model, version, or scheme field directly changes $K_{m,v,s}$. Consequently, packages issued under different contexts do not reuse the same final encryption key.

4.4 Hardware-Bound Functional Ownership Evidence

A side-benefit of deriving $K_{m,v,s}$ directly from K_{master} and the authenticated DID , OID , MID , v , and s context is hardware-bound functional evidence that does not require a verifier to access the vendor's protected enrollment record. Successful authorized execution demonstrates that the signed package is functionally bound to the stated device and public owner identifier, but it neither authenticates the claimant's real-world identity nor constitutes a cryptographic proof of legal ownership.

Verification protocol. This verification process is designed for two direct-execution scenarios: local self-verification by an end-user, and formal arbitration by a Trusted Third Party (TTP). To test a claimed binding at the edge device, the verifier first checks the provider signature over the package and its DID , OID , MID , v , and s metadata using the authenticated provider public key described in Sec. 4.1. The device reconstructs K_{master} through the fuzzy extractor and then directly derives $K_{m,v,s}$ exactly as specified in Sec. 4.1. It attempts to decrypt the package with $K_{m,v,s}$, after which the verifier runs a small accuracy benchmark. Based on the empirical outcome, we define three statistical verdicts:

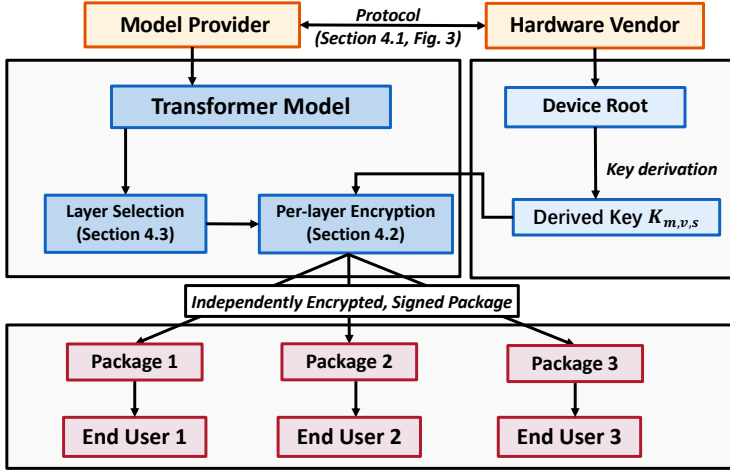


Fig. 6. Complete model protection workflow. The model provider encrypts each package using a device-owner-model-version-scheme key derived from the corresponding PUF root, allowing the same plaintext model to be distributed as independently bound encrypted packages.

- If the resulting accuracy matches the published Acc_0 (e.g., 80.33% for ViT-B on ImageNet), accept the functional binding only if it is at least a preregistered threshold τ_{hi} close to Acc_0 .
- If the accuracy collapses to random-guess level ($\approx 0.1\%$), reject the functional binding when accuracy is at most τ_{lo} .
- Return an inconclusive verdict between the two thresholds.

In local provenance checks, an end-user may simply utilize a public test set to ensure the model executes correctly under the claimed *OID*. However, in formal copyright disputes, a TTP acts as the arbitrator. The TTP executes the obfuscated model within a physically controlled environment using a private, unreleased benchmark dataset; the claimed device-package-*OID* binding is accepted when the corresponding key restores the benchmark accuracy. Preregistering the (τ_{lo}, τ_{hi}) bounds and utilizing private benchmark sets in TTP scenarios permits a statistical false-accept bound against surrogate models.

Binding security and decision errors. Under the pseudorandomness assumption of HKDF and the secrecy of K_{master} , public knowledge of *OID* does not enable a party to derive $K_{m,v,s}$. A copied package on another device starts from a different K_{master} and therefore derives a different $K_{m,v,s}$. The public *OID* labels the binding but does not authenticate whoever presents it; a verifier that must associate the binding with a legal person or organization needs an independent identity-authentication procedure. This key binding also does not make the accuracy threshold itself a proof system. False negatives can arise from PUF reconstruction failure, a corrupted package, nondeterministic execution, or dataset shift. False positives can arise from statistical variance on an undersized benchmark, an independently functional surrogate model that mimics target accuracy regardless of decryption, or a task-specific weak obfuscation vulnerability where an incorrect key derivation fails to degrade accuracy below τ_{hi} . We therefore claim hardware-bound functional evidence, not an unconditional cryptographic ownership proof.

5 Experiments

In this section, we validate our approach on vision transformers [29], including ViT and several variants: Swin Transformer [72], DeiT [115], and BeiT [7]. We utilize variants with differing model sizes, input resolutions, and quantization levels. Table 3 details these models, their original accuracy (Acc_0), and the lossless decrypted accuracy (Acc_{dec}). Unless otherwise stated, the primary GPU experiments were conducted on a Linux server equipped with NVIDIA GeForce RTX 4090 GPUs, running CUDA 12.8, Python 3.10.12 with PyTorch 2.8.0. Custom GPU kernels are implemented using Triton 3.4.0. Model training and evaluation rely on the HuggingFace Transformers library (v4.56.1). To test transfer beyond vision and empirical scaling, we additionally evaluate RoBERTa on SST-2, pretrained OPT-1.3B and OPT-6.7B on WikiText-103, and Wav2Vec 2.0 speech recognition on LibriSpeech; latency is the median of five trials. Vision models and RoBERTa use batch size 64, while the OPT LLMs and Wav2Vec 2.0 use batch size 16.

Crucially, the table also demonstrates the efficiency of our obfuscation strategy. The column Target Layers lists the specific layer indices identified by our Top- K strategy that are required to reduce the model's performance to near random-guess levels ($< 0.15\%$). For example, encrypting just a single layer (Layer 1) in ViT-L is sufficient to drop accuracy to 0.10%, while ViT-B requires only two layers (Layers 0 and 10). The final column, Acc' , reports this degraded accuracy, confirming that our method effectively neutralizes the model with minimal encryption overhead.

Table 3. Model specifications and protection effectiveness (validated on ImageNet). Acc_0 and Acc_{dec} denote the original and authorized decrypted accuracies, respectively. The **Target Layers** column indicates the specific layers encrypted under the Top- K strategy to rapidly degrade performance to the random-guess level shown in Acc' .

Model	Params (M)	Acc_0 (%)	Acc_{dec} (%)	Target Layers	Acc' (%)
ViT-B	86	80.33	80.33	(0, 10)	0.10
ViT-B-384	86	83.91	83.91	(0, 11)	0.09
ViT-L	307	81.98	81.98	(1)	0.10
ViT-L-384	307	84.96	84.96	(1)	0.07
Swin-B*	88	85.14	85.14	(2-9, 2-8)	0.10
Swin-B-384*	88	86.44	86.44	(2-9, 3-1)	0.08
BeiT-B	86	84.54	84.54	(2, 9)	0.12
DeiT-S	22	77.1	77.1	(7, 0)	0.09
DeiT-B	86	80.20	80.20	(8, 0)	0.10
ViT-B-8b	86	80.24	80.24	(6)	0.11

*For Swin Transformer, indices are denoted as (Stage-Layer). “S” “B” “L” denote the “Small” “Base” “Large” size of the model. The input resolution is by default “ 224×224 ,” and is “ 384×384 ” for others. “8b” denotes 8-bit quantization of model weights.

Concrete PUF realization. To empirically evaluate hardware-bound key generation across multiple devices and environmental corners, we utilize the public Arduino UNO SRAM-PUF corpus [93]. This dataset contains 15 nominal measurements for reliability-aware enrollment alongside responses from 10 boards across -22.5 – 69°C and 8 boards across 3.8 – 6.2 V. Across the temperature sweep, the raw bit error rate (BER) averages 3.18% and reaches 44.76% in the worst-case outlier. To support reliable 128-bit key reconstruction, we adopt established fuzzy-extractor constructions [16, 25, 57]. The extractor employs repetition-5 combined with a shortened BCH code capable of correcting 63 residual errors, followed by SHA-256 for privacy amplification. The implementation consumes 3,800 of the available 8,192 PUF bits as a conservative extraction input budget and stores 475 bytes

of helper data; deployment-specific entropy characterization is still required to verify the 256-bit conditional-min-entropy requirement formalized in Sec. 2.3. Without using the environmental test responses during enrollment, the extractor successfully reconstructs 379/380 temperature-corner keys (99.74%) and 200/200 voltage-corner keys (100%), while rejecting all 146 cross-device reconstruction attempts. The single rejected temperature response, which is the 44.76%-BER outlier, exceeds the correction budget and therefore safely fails closed, illustrating why a deployment must characterize its operating range and re-enroll a device that drifts beyond it. We stress that ChaosFormer is agnostic to this specific instantiation: other PUF families, such as a controlled Arbiter PUF on an FPGA or a DRAM-latency PUF on a memory-rich platform, remain possible alternatives, provided the chosen primitive satisfies the conditional-min-entropy and reconstruction requirements stated in Secs. 2.3 and 6.4.

Edge-GPU benchmarking. The overhead measurements in Sec. 5.3 are reported on an RTX 4090 to isolate the cost of the encryption kernel itself. To verify that the conclusions transfer to realistic edge hardware, we also benchmark the same workload on an NVIDIA Jetson Orin Nano, which is closer to the class of accelerator likely to be deployed with a paired SRAM-PUF root of trust. The Jetson’s lower memory bandwidth and smaller SM count slow down the baseline forward pass proportionally more than they slow down the memory-bound encryption kernel, so the relative overhead on the Jetson is, in fact, *lower* than on the RTX 4090.

5.1 Effectiveness Test

The primary goal of ChaosFormer is to render the model unusable without the correct PUF-derived key (model-version-scheme key) while ensuring lossless performance for authorized users. As shown in Table 3, the authorized accuracy (Acc_{dec}) matches the original accuracy (Acc_0) exactly, confirming that our reversible obfuscation induces no performance degradation.

To evaluate the protection strength (i.e., the degradation of accuracy Acc' without the key) of ChaosFormer, we investigate three layer selection strategies for encryption mentioned in Sec. 4.3 with encryption budgets of $K = 4$ and $K = 6$ layers. As detailed in the “Acc.” column of Table 6, all three strategies successfully degrade the model performance to random-guess levels (approx. 0.1%) regardless of K . For instance, with $K = 6$, the accuracy drops to 0.09% (Last- K), 0.10% (Top- K), and 0.10% (Random- K) for ViT-B. Similar effectiveness is observed for DeiT-S, where the Top- K strategy reduces accuracy to 0.05%. This confirms that partial encryption of critical weights is sufficient to completely disable the model’s inference capability.

Cross-domain and billion-parameter validation. Table 4 extends our evaluation beyond vision to encompass NLP encoders (RoBERTa), billion-parameter LLM decoders (OPT-1.3B, OPT-6.7B), and speech recognition (Wav2Vec 2.0) [71, 138]. Across all 872 SST-2 validation examples [121], 128 non-overlapping 256-token WikiText-103 sequences [78], and 2,620 LibriSpeech test-clean utterances [88], authorized execution with the correct key strictly preserves the original accuracy (Acc_0), perplexity (PPL), and Word Error Rate (WER). Conversely, executing the Base scheme ($K = 6$) with a wrong key substantially degrades utility: RoBERTa accuracy approaches binary random guessing at 50.92%, OPT-6.7B perplexity (lower the better) increases from 19.21 to 9,964.86, and Wav2Vec 2.0 WER (lower the better) reaches 100%. The Secure variant (ChaosFormer-S) produces non-finite logits across these modalities (denoted as “Invalid”).

5.2 Robustness Test

5.2.1 Retraining and Fine-tuning Attacks. A sophisticated adversary with white-box access to the encrypted model might attempt to recover its functionality by retraining the weights. We evaluate robustness against two distinct attack vectors using limited data assumptions:

Table 4. Cross-domain effectiveness and billion-parameter scaling. “Invalid” indicates that unauthorized execution produces non-finite logits.

Model/task	Scheme	K	Metric	Original	Wrong key	Authorized / overhead
RoBERTa/SST-2	Base	6	Acc. (%)	94.04	50.92	94.04 / 1.10×
RoBERTa/SST-2	Secure	12	Acc. (%)	94.04	Invalid	94.04 / 1.43×
OPT-1.3B/WikiText	Base	6	PPL	26.56	12,125.61	26.56 / 1.08×
OPT-1.3B/WikiText	Secure	6	PPL	26.56	Invalid	26.56 / 1.09×
OPT-6.7B/WikiText	Base	6	PPL	19.21	9,964.86	19.21 / 1.07×
OPT-6.7B/WikiText	Secure	6	PPL	19.21	Invalid	19.21 / 1.14×
Wav2Vec2/LibriSpeech	Base	6	WER (%)	3.37	100.00	3.37 / 1.05×
Wav2Vec2/LibriSpeech	Secure	12	WER (%)	3.37	Invalid	3.37 / 1.22×

- (1) **Same-Application Retraining:** The attacker retrains the encrypted model on ImageNet using 20% of the original training data.
- (2) **Domain-Transfer Fine-tuning:** The attacker fine-tunes the encrypted model (after the 20% ImageNet retraining) on downstream datasets, as shown in Table 5. In particular, we utilize the “product” subset for training and the “real world” subset for testing in the OfficeHome dataset; meanwhile, we choose the “Clipart” subset of DomainNet. Both are used to test the robustness of the model to distribution shift.

Table 5. Dataset statistics used in robustness evaluation.

Dataset	Train	Validation	Test	#Classes
CIFAR-100	45,000	5,000	10,000	100
Caltech-256	24,482	3,060	6,084	256
CUB-200	5,994	2,997	2,997	200
ImageNet-100	130,000	5,000	5,000	100
OfficeHome	43,439	4,000	4,000	65
DomainNet	413,678	52,041	52,041	345

Baselines. To quantify the value of the stolen IP, we compare the attacker’s results against three baselines representing the upper bound and the “build-vs-steal” alternatives:

- **Original Model (Origin):** Fine-tuning the original, unencrypted pre-trained model to represent the upper bound of performance.
- **Same Architecture (RandInit):** Training the same transformer architecture from scratch using standard initialization under the attacker’s exact data and compute budget. This serves as the direct, fair baseline for evaluating retained IP value.
- **EfficientNet-b0 (RandInit):** Training a highly efficient standard CNN [111] ($\approx 5.3\text{M}$ params) from scratch to represent a low-cost alternative.

Training Details. We standardize our training configurations across all models to ensure rigorous comparison. Based on an initial empirical comparison of SGD, Adam, and AdamW optimizers, we selected AdamW as it consistently yielded the most stable convergence for both the Transformer and CNN architectures. We subsequently conducted a grid search over learning rates (LR) and weight decay (WD) to establish optimal default configurations. For the initial retraining phase on the 20% ImageNet subset, models are trained using AdamW with a batch size of 128, a WD of 0.05,

and a peak LR between 3×10^{-5} and 2×10^{-4} (scaled by model capacity and initialization state). This is regulated by a cosine annealing schedule with 3 to 5 warmup epochs. To maximize the attacker’s recovery potential, we employ a strong regularization recipe including RandAugment, CutMix ($\alpha \in [0.5, 1.0]$), MixUp ($\alpha \in [0.2, 0.8]$), and label smoothing (0.05–0.10). Because the quality of the recovered weights severely impacts downstream transferability, we conduct three independent retraining attempts with different random seeds and strictly select the checkpoint with the *best* validation accuracy for subsequent attacks. During downstream fine-tuning, models are optimized for 10 to 30 epochs using AdamW (LR $\in [1 \times 10^{-4}, 2 \times 10^{-4}]$, WD 0.05, batch size 64 or 128) guided by a cosine schedule with a 10% step warmup and global gradient clipping at 1.0. All fine-tuning experiments are run with three random seeds, and the *average* accuracy is reported. The EfficientNet-b0 baseline follows an identical hyperparameter search and multi-seed evaluation protocol.

Table 6. Robustness analysis of ChaosFormer on ViT-B and DeiT-S. We report the accuracy of the encrypted model, accuracy after retraining on 20% of ImageNet, and downstream transfer learning performance for $K = 4$ and $K = 6$ encrypted layers. *RandInit* baselines train the identical architecture from scratch. “IN-100” denotes the ImageNet-100 dataset.

Config.	Strategy	Init.	Retrain	Downstream Fine-tuning Acc. (%)					
		Acc.	IN-20%	C100	Cal	CUB	IN100	OH	DN
ViT-B	Origin	80.33	—	92.31	93.61	84.33	94.32	82.63	81.04
DeiT-S	Origin	77.14	—	92.12	87.23	72.05	87.15	72.04	76.16
ViT-B	RandInit	—	36.19	70.14	60.18	44.34	65.14	36.84	52.95
DeiT-S	RandInit	—	35.11	69.59	59.07	43.23	63.72	34.45	51.10
EffNet-b0	RandInit	66.12	—	83.22	82.98	74.04	88.16	59.72	72.84
Encryption with $K = 4$ Layers									
ViT-B	Last-K	0.15	71.17	87.99	87.30	70.33	91.52	71.43	75.97
ViT-B	Top-K	0.09	66.82	88.45	85.26	70.59	90.26	69.68	73.20
ViT-B	Rand-K	0.15	64.59	86.42	83.08	65.88	87.16	65.25	69.58
DeiT-S	Last-K	0.13	69.37	84.47	84.42	69.09	90.54	66.19	71.17
DeiT-S	Top-K	0.08	60.77	81.29	77.07	49.31	85.52	53.80	64.24
DeiT-S	Rand-K	0.14	63.14	82.48	79.29	48.81	85.54	57.68	65.47
Encryption with $K = 6$ Layers									
ViT-B	Last-K	0.09	65.16	86.37	84.45	68.78	89.36	65.30	73.03
ViT-B	Top-K	0.10	62.72	86.06	82.27	64.62	88.26	64.98	70.99
ViT-B	Rand-K	0.10	57.39	83.35	78.04	59.94	84.78	54.53	64.85
DeiT-S	Last-K	0.14	61.75	82.08	79.94	61.72	87.82	59.22	67.52
DeiT-S	Top-K	0.05	51.10	75.37	68.28	43.56	80.28	42.87	57.31
DeiT-S	Rand-K	0.10	57.90	79.96	75.53	42.03	83.82	52.21	62.73

Abbreviations: Init.: Initial; IN-20%: ImageNet (20%); C100: CIFAR-100; Cal: Caltech-256; CUB: CUB-200; OH: OfficeHome; DN: DomainNet; EffNet-b0: EfficientNet-b0; Rand-K: Random-K.

Analysis of ChaosFormer. Table 6 presents the results for the base scheme. We observe that increasing the encryption budget from $K = 4$ to $K = 6$ significantly degrades the attacker’s ability to recover performance. While greedy Top-K reliably minimizes initial inference accuracy, Random-K is heterogeneous: a favorable draw can match Top-K under retraining without running or exposing

a deterministic greedy selection path, but not every random draw is uniformly stronger. Critically, regarding downstream tasks, while the stolen models retain some transferability compared to training the identical architecture from scratch (*RandInit ViT-B*), their recovered performance suffers massive degradation compared to the original unencrypted model. On complex domain-shift tasks like DomainNet or OfficeHome, the attacker loses substantial accuracy (e.g., dropping from 82.63% to 64.98% on OfficeHome for ViT-B with $K = 6$ Top- K). Furthermore, as the encryption budget increases, the stolen transformer’s utility diminishes to the point where an attacker is better off simply training a lightweight, low-cost CNN baseline (*EfficientNet-b0*) from scratch, which achieves superior performance on DomainNet (72.84%) compared to the retrained $K = 6$ models.

Analysis of ChaosFormer-S. While the base ChaosFormer prevents direct usage, adversaries possessing a public base model might attempt to recover weights via statistical alignment (as discussed in Sec. 3). To counter this, ChaosFormer-S (1) incorporates a diffusion mechanism that renders encrypted weights computationally indistinguishable from uniform under the PRF assumption and (2) applies encryption to all layers to avoid analysis of any of them. As shown in Table 7, the retraining performance of ChaosFormer-S is functionally worse than training the identical architecture (or EfficientNet-b0 in Table 6) from scratch across all tasks.

Standard random initializations (e.g., Xavier or Kaiming) are carefully designed to maintain variance across layers to facilitate gradient flow. In contrast, our diffusion-based encryption produces encoded weights that are computationally indistinguishable from uniform bit strings under the PRF assumption. We hypothesize that these encoded weights disrupt the variance properties needed for stable gradient flow, producing a jagged optimization landscape with vanishing or exploding gradients. Under this interpretation, an attacker must first “unlearn” the hostile noise before acquiring useful features, resulting in slower convergence and lower final accuracy than simply starting from a properly initialized network.

Table 7. Robustness analysis of ChaosFormer-S on ViT-B and DeiT-S. The secured model (encrypting *all* layers) is compared against a random-initialization baseline. Its consistently lower recovered accuracy shows that the encrypted checkpoints provide less empirical utility than starting from the corresponding random initialization in these attacks.

Config.	Enc.	Retrain	Downstream Fine-tuning Acc. (%)					
	Acc.	IN-20%	C100	Cal	CUB	IN100	OH	DN
ChaosFormer-S (ViT-B)	0.10	27.36	67.29	52.73	36.88	61.18	29.22	44.54
Rand (ViT-B)	—	36.19	70.14	60.18	44.34	65.14	36.84	52.95
ChaosFormer-S (DeiT-S)	0.10	32.08	65.31	52.99	37.33	62.20	26.88	40.67
Rand (DeiT-S)	—	35.11	69.59	59.07	43.23	63.72	34.45	51.10

Abbreviations: Rand: RandInit Baseline; Enc.: Encrypted; IN-20%: ImageNet (20%); C100: CIFAR-100; Cal: Caltech-256; CUB: CUB-200; IN100: IN-100; OH: OfficeHome; DN: DomainNet.

5.2.2 Key Recovery Attacks. We further analyze a conservative scenario in which the attacker is granted the exact identities of the layers encrypted by ChaosFormer. The admitted threat model includes end-to-end inference-latency observations, but excludes the privileged co-location and cache/runtime-memory traces that could reveal individual layer operations [96]. We therefore grant the layer identities directly and evaluate whether this knowledge enables key recovery, without treating the excluded acquisition channels as part of the claimed protection.

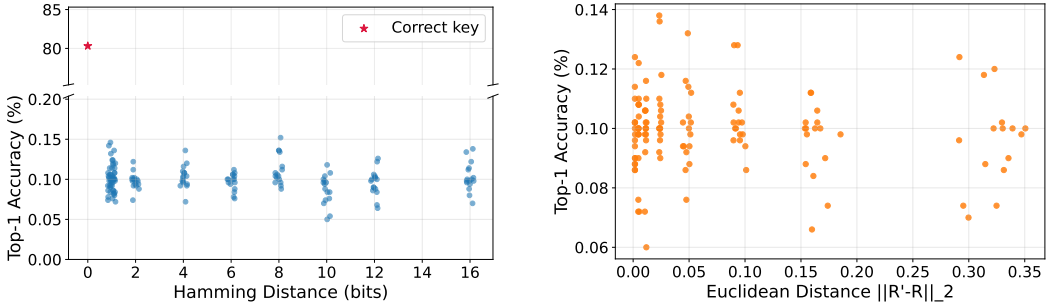


Fig. 7. Sensitivity analysis of the system. Left: Sensitivity to bit flips in the model-encryption key. Right: Diagnostic wrong-root-key behavior when simulated PUF-response perturbations are passed to key derivation without the production fuzzy-extractor acceptance check.

A direct attack would involve attempting to recover the secret keys. The combination of the ACM parameters (p, q) for attention layers and the seeds for the FFN permutations presents a vast search space that makes exhaustive guessing impractical. Because ACM and the Knuth Shuffle are discrete permutations, they also preclude direct gradient-through-permutation attacks. For the base scheme, an incorrect key empirically destroys task utility in our evaluated configurations, but the permuted tensor is not cryptographically indistinguishable from random. The computational indistinguishability claim applies only to ChaosFormer-S under the PRF and key-secrecy assumptions.

Our experiments in Fig. 7 demonstrate the wrong-key utility collapse. We evaluated a ViT-B model with $K = 6$ encrypted layers (applying obfuscation to both Attention and FFN weights) and tested decryption by bit-flipping the correct model-encryption key and, as a diagnostic, perturbing the simulated PUF response while bypassing the production fuzzy-extractor acceptance check. The plots show that the correct key yields full accuracy ($\approx 80\%$), whereas either path to a mismatched key collapses accuracy to approximately 0.1%. In the deployed hierarchy, environmental variation within the correction budget reconstructs the exact K_{master} and therefore the exact $K_{m,v,s}$; variation beyond that budget is rejected before model decryption. The right panel therefore characterizes the consequence of an incorrect root, not the PUF reliability mechanism. This step-function behavior supplies no useful accuracy feedback in the evaluated setting. This distinction matters because Basak et al. [9] recover a PUF-controlled BNN key bit-by-bit precisely by observing accuracy improvements from individually corrected column swaps.

Defense in depth across layers. Multi-layer encryption is not a redundancy; it is what makes the avalanche effect operationally useful. Suppose, hypothetically, that the attacker were extraordinarily lucky and recovered the key for a single encrypted layer — whether through guessing, side-channel leakage, or some unforeseen weakness of the underlying primitive. Even so, the remaining $K - 1$ encrypted layers continue to scramble the forward pass, and the resulting model accuracy stays at the random-guess baseline. Concretely, we ran the experiment on three base-sized models (ViT-B, DeiT-B, BeiT-B), each with $K = 6$ encrypted layers under ChaosFormer, and decrypted exactly one layer at a time. The accuracy change ranged from -0.04% to $+0.04\%$ across all 18 runs — in other words, no statistically detectable signal. Thus, unlike the differential oracle exploited against the PUF-based BNN [9], a correct single-layer guess produces no measurable feedback across our 18 trials while the other five layers remain locked. Layer-by-layer brute force consequently lacks an

empirical fitness signal in this configuration. We treat this as defense-in-depth evidence rather than a proof against every adaptive or side-channel attack.

5.3 Inference Overhead

Managing the inference overhead is crucial for practical deployment on edge devices. Our experiments involve a dynamic, layer-by-layer de-obfuscation process: for each encrypted layer, the model decrypts the weights, executes the layer operation, and immediately re-encrypts (or discards) the plaintext weights before proceeding. This bounds the simultaneously resident plaintext and shortens its lifetime; as stated in Sec. 3, it is exposure reduction rather than protection against a privileged runtime-memory adversary. To account for hardware-bound initialization, we separate root-key reconstruction from the recurring tensor pipeline. The measured SRAM-PUF software path takes a supplied noisy response R'_{dev} and helper data P through repetition and BCH decoding, integrity verification, and SHA-256 privacy amplification to reconstruct K_{master} ; physical SRAM sensing latency is construction-specific and is not emulated by the accelerator kernels. This one-time path costs $31.3 \mu s$ at the median and $57.0 \mu s$ at the 95th percentile. The subsequent HKDF derivations of $K_{m,v,s}$, K_i , and the per-tensor diffusion records are included in model loading and cipher setup; recurring protected inference includes the on-the-fly ChaCha20 keystream generation and tensor transforms, but not the HKDF execution. The root and descendant keys remain in the protected key path, while plaintext-layer exposure follows the runtime boundary stated above.

We evaluate the overhead on both CPU and GPU platforms using a ViT-B model (Batch Size = 64). We analyze two variants of our framework: the permutation-only ChaosFormer (Base) and the diffusion-enhanced ChaosFormer-S (Secure). We analyze two factors that affect the runtime overhead: the number of ACM iterations (n) and the number of encrypted layers. When studying the former, we fix the number of encrypted layers at 6; when studying the latter, we fix the ACM iterations at 5. This maximally isolates the effects of each factor.

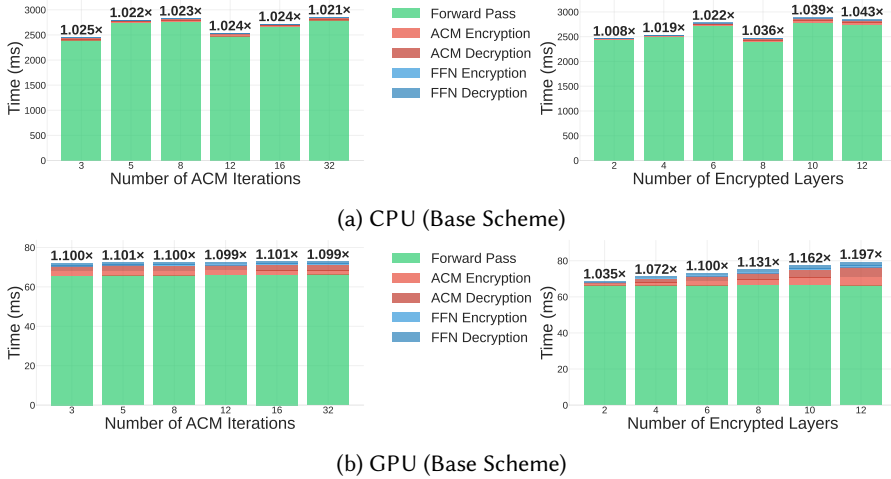


Fig. 8. **ChaosFormer Overhead.** The stacked bars show the time breakdown in milliseconds.

5.3.1 *ChaosFormer (Base Scheme).* Fig. 8 presents the overhead breakdown for the base scheme.

- **Impact of Iterations:** As shown in the left subplots of Figs. 8a and 8b, increasing the number of ACM iterations from 3 to 32 has a negligible impact on total inference time. On the GPU, the

overhead remains constant at approximately 1.10 $\times$, indicating that our optimized parallel ACM implementation is highly efficient.

- **Impact of Layer Count:** The right subplots demonstrate a linear relationship between the number of encrypted layers and overhead. On the CPU, the overhead is extremely low; encrypting 12 layers results in only a 1.04 $\times$ slowdown (Fig. 8a). On the GPU, where the baseline forward pass is significantly faster, the relative cost of memory-bound encryption operations is higher but still practical, reaching 1.20 $\times$ for 12 layers (Fig. 8b).

5.3.2 ChaosFormer-S (Secure Scheme). Fig. 9 evaluates the overhead of the secure variant, which includes the diffusion mechanism to thwart statistical attacks.

- **CPU Performance:** As seen in Fig. 9a, the addition of the diffusion step incurs minimal penalty on the CPU. The maximum overhead for 12 encrypted layers increases slightly from 1.04 $\times$ (Base) to 1.07 $\times$ (Secure). This confirms that the diffusion operation is computationally inexpensive relative to the matrix multiplications in the Transformer forward pass.
- **GPU Performance:** On the GPU (Fig. 9b), the overhead for 12 layers increases to 1.26 $\times$. While the relative overhead is higher due to the rapid execution of the baseline model, the absolute latency penalty remains in the millisecond range.

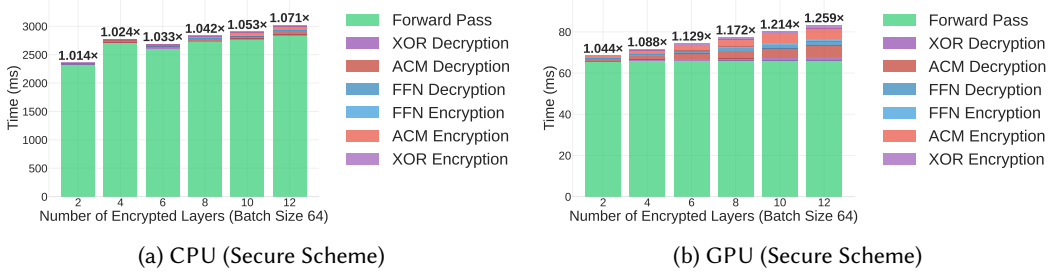


Fig. 9. **ChaosFormer-S Overhead.** These figures quantify the cost of adding the diffusion mechanism.

Edge-GPU validation on the Jetson Orin Nano. To address the concern that RTX-class measurements may not transfer to realistic edge hardware, we re-run the secure-scheme overhead sweep on an NVIDIA Jetson Orin Nano. Fig. 10 reports the result. The relative overhead is, in fact, *lower* than on the RTX 4090: even at $K = 12$ encrypted layers, it remains at 1.058 $\times$, compared to 1.26 $\times$ on the RTX 4090. This is consistent with the analysis at the start of Sec. 5 — the baseline forward pass is the bottleneck on memory-bandwidth-constrained edge accelerators, so the absolute cost of the encryption kernel constitutes a smaller fraction of the total runtime. This result demonstrates low relative overhead for the SRAM-PUF-paired Jetson-class deployment targeted by our threat model.

Observation and Summary. To provide a precise numerical baseline, Table 8 offers a concise snapshot of the baseline, ChaosFormer, and ChaosFormer-S latencies for a fixed configuration (6 encrypted layers, batch size 64) across representative platforms. Beyond this fixed configuration, the dynamic sweeps demonstrate robust scalability: The RTX sweep measures at most 26% overhead with 12 encrypted layers, while the CPU overhead remains approximately 7% or lower. These results support the practicality of ChaosFormer for latency-sensitive edge applications.

Memory footprint. A natural follow-up question is whether the encryption pipeline inflates memory consumption to a level that conflicts with edge deployment. We instrument the CUDA allocator (`torch.cuda.max_memory_allocated`) to measure the transient peak memory of the

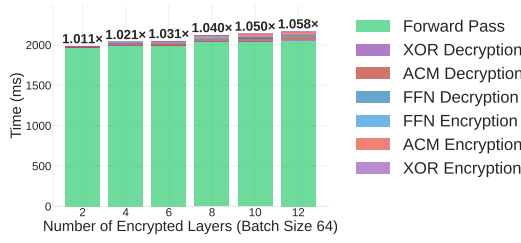


Fig. 10. **ChaosFormer-S overhead on NVIDIA Jetson Orin Nano** (Batch Size 64). At $K = 12$ encrypted layers the relative overhead is 1.058 $\times$, lower than the 1.26 $\times$ measured on the RTX 4090.

Table 8. ViT-B latency at fixed $K = 6$ and batch size 64 on different platforms.

Platform	Baseline (ms)	ChaosFormer (ms; overhead)	ChaosFormer-S (ms; overhead)
Xeon 8255C CPU	2,540	2,612 (1.028 $\times$)	2,704 (1.064 $\times$)
RTX 4090 GPU	141.6	144.8 (1.023 $\times$)	146.9 (1.037 $\times$)
Jetson Orin Nano	1,975	2,005 (1.015 $\times$)	2,049 (1.037 $\times$)

protected pipeline against the unprotected baseline on ViT-B and observe two properties. First, transient peak memory is *independent* of the number of encrypted layers: encryption operates layer by layer, the diffusion keystream is generated on-the-fly inside the Triton kernel rather than materialized as a separate tensor (Sec. 4.2.3), and the per-layer key record is a small constant. Second, in absolute terms, the overhead is small — roughly 3 MiB for ChaosFormer and 19 MiB for ChaosFormer-S, regardless of the protection budget. At a representative batch size of 64, where activations occupy ≈ 518 MiB, this fixed working set amounts to a $\leq 3.5\%$ memory tax.

Overhead scaling on billion-parameter and cross-domain models. Let $\mathcal{S}$ be the protected layer set, $\mathcal{L}$ the transformer weight matrices, and $P_{\mathcal{S}}$ and $P_{\mathcal{L}}$ their respective numbers of weights. De-obfuscating and re-obfuscating the protected matrices costs $T_{\text{protect}} = O(P_{\mathcal{S}})$, whereas dense projections over B sequences of length L_{seq} require $T_{\text{compute}} = \Omega(BL_{\text{seq}}P_{\mathcal{L}})$ even before the additional attention computation. The relative work is therefore $O(P_{\mathcal{S}}/(BL_{\text{seq}}P_{\mathcal{L}}))$: parameter growth alone does not increase it when the protected fraction is fixed, and token reuse amortizes the protection work. Because protected layers are processed sequentially, the additional workspace is $O((\max_{i \in \mathcal{S}} |W_i|))$ rather than the sum of all protected weights.

The measurements follow this scaling behavior. For the official 6.658B-parameter OPT-6.7B checkpoint, protecting six layers (1.208B weights) requires a peak of 16.42 GB of GPU memory and incurs 1.07 $\times$ /1.14 $\times$ Base/Secure prefill latency at batch size 16. Across batch sizes 8, 16, 32, and 64, the Base overhead decreases from 1.15 $\times$ to 1.02 $\times$ on OPT-1.3B and from 1.16 $\times$ to 1.03 $\times$ on OPT-6.7B; the OPT-6.7B Secure overhead similarly decreases from 1.36 $\times$ to 1.01 $\times$. At their primary batch sizes, the Base/Secure overheads are 1.10 $\times$ /1.43 $\times$ for RoBERTa and 1.05 $\times$ /1.22 $\times$ for Wav2Vec 2.0. RoBERTa’s larger Secure ratio reflects its short encoder workload and all-12-layer protection budget. These results establish the predicted amortization behavior in the evaluated dense FP16 prefill regime.

5.4 Comparison

To highlight the advantages of our proposed method, this section conducts a comparative analysis with existing works, evaluating both the encryption module overhead in isolation (Sec. 5.4.1) and the qualitative IP protection features (Sec. 5.4.2).

5.4.1 Comparison with Common Encryption Methods. Conventional methods for protecting on-device models often encrypt the entire model file using schemes like AES, the Tiny Encryption Algorithm (TEA), and basic bitwise operations (XOR), and then decrypt it fully into memory before inference, as observed in many Android apps [26]. Block ciphers such as AES are optimized for general data and CPU execution, not for the structure or redundancy of neural network weights. In contrast, our proposed obfuscation can decrypt weights one layer at a time during inference and re-encrypt them immediately after use. This avoids simultaneous full-model plaintext residency, although a privileged runtime adversary could still accumulate layers across executions. ACM operations are also highly parallelizable for GPU acceleration.

We benchmarked our method against AES (with AES-NI support), TEA, and XOR (without and with GPU acceleration), evaluating the additional wall-clock time for a single round of decryption and encryption of a ViT-B model (330 MB) and a ViT-L model (1161 MB). CPU methods are tested with an Intel Xeon Platinum 8255C @ 2.50GHz with 32 cores. For GPU-based methods, only the time spent on the GPU (excluding CPU-GPU communication time) is reported. Table 9 shows that our proposed method achieves the lowest time overhead on both CPU and GPU and is faster than different modes of AES, even with special hardware acceleration. In particular, our previous experiments in Sec. 5.1 show that on larger models, our method requires even fewer layers to achieve satisfactory accuracy reduction; it therefore scales more gracefully than methods like the naive XOR cipher while providing stronger security.

Table 9. Encryption/decryption runtimes. Evaluations are carried out on a 330 MB ViT-B model and a 1161 MB ViT-L model. Each experiment is carried out 5 times and the average is reported. For ACM, n is set to 3, and the number of encrypted layers is 6 for ViT-B and 7 for ViT-L.

Method	Time (s) on ViT-B	Time (s) on ViT-L	Device
AES-CBC	2.30	9.02	CPU
AES-CTR	1.34	5.17	CPU
AES-GCM	1.45	5.18	CPU
TEA	19.63	69.51	CPU
XOR Cipher	3.72	13.47	CPU
Our Method (CPU)	1.19	2.58	CPU
XOR Cipher (GPU)	0.043	0.22	GPU
Our Method (GPU)	0.044	0.051	GPU

5.4.2 Comparison with Other IP Protection Approaches. Table 10 compares our proposed work with representative IP protection approaches in the literature. In particular, the “Retrain” column denotes whether the model owner needs to retrain the models with additional regularizers or customized datasets, which often impede convergence and prolong the training process, while also lacking scalability for a substantial number of users. In contrast, like all encryption-based schemes, our approach does not require retraining or degrade model performance (“Acc. Loss” column).

TransLinkGuard (TLG) [67] addresses on-device transformer deployment by securing layers with weight matrix permutations within a TEE. However, TLG is ill-suited for real-time edge deployment, as its mask-offload-restore pipeline requires the TEE to repeatedly compute noise effects mW (matrix multiplication), which incurs prohibitive latency and memory overhead [100]. Moreover, their obfuscation scheme is purely linear, and attackers with access to permuted weights and the original public model might align columns to recover the permutations [123]. For methods that do not explicitly report inference-time overhead [32, 68], we evaluate their publicly available

implementations under the same hardware and software configuration as ours, using standard ResNet-18 models with ImageNet-scale inputs (224×224) and a consistent batch size (e.g., 64). While these measurements demonstrate that ChaosFormer achieves highly practical and low-overhead execution within its evaluated workloads, we explicitly note that this is not a normalized, security-equivalent ranking. The evaluated baselines target different model families, rely on different trust anchors, and use a mixture of author-reported and reproduced measurements. We therefore compare numerical overhead only where the workload and accounting are identified, and use Table 11 for qualitative security positioning.

Table 10. Operational comparison of IP protection methods.

Work	Type	Mechanism	Retrain	Target	Infer. Overhead	Acc. Loss
Uchida et al. [117]	Passive	WB Watermarking	✓	CNN	None	✓
Adi et al. [2]	Passive	BD Watermarking	✓	CNN	None	✓
FreeMark [22]	Passive	Keyed Fingerprinting	×	CNN	None	×
M-LOCK [97]	Active	BD Access Control	✓	CNN	None	✓
DeepIPR [32]	Active	Passport Verification	✓	CNN	Moderate (+50%)	✓
NNLock [3]	Active	Wt. Encryption	×	CNN	Moderate (+100%)	×
Phantom [6]	Active	Arch. Obfusc., TEE	✓ [#]	CNN	Moderate (+250% ^{h1})	✓ ^{h2}
ChaoW [68]	Active	Wt. Encryption	×	CNN	Moderate (+105%)	×
TLG [67]	Active	Wt. Permutation, TEE	×	TF	High [100]	×
LoRO [129]	Active	Wt. Permutation, TEE	×	TF	Moderate (+149%)	×
ChaosFormer	Active	Wt. Encryption, PUF	×	TF	Low (+2–10%)*	×
ChaosFormer-S	Active	Wt. Encryption, PUF	×	TF	Low (+7–26%)*	×

Abbreviations: WB: White-box; BD: Backdoor; Wt.: Weight; Exec.: Execution; Arch. Obfusc.: Architecture Obfuscation; CNN: Convolutional Neural Network; TF: Transformer; Infer.: Inference. [#]Includes neural architecture search (≈ 26 h for ResNet-18 on CIFAR-10, single A6000). ^{h1}Includes obfuscation layer, TEE latency, and TEE–GPU communication; with ResNet-18, SGX2, Top-3 strategy. ^{h2}No direct accuracy loss, but uses 8-bit quantization for TEE–GPU data transfer. *Based on encrypting 6 layers. Values mix author-reported measurements and our reproductions and should not be read as a normalized ranking across model families or threat assumptions.

Table 11. Security properties of IP protection methods. “Unauthorized inference” means a copied protected checkpoint is nonfunctional on unauthorized hardware. “Chaotic dynamics” denotes a mathematically analyzed dynamical property.

Work	Target	Required trust/access	Protection	Formal claim
Uchida/Adi [2, 117]	CNN	Owner secret + model access/query	Ownership verification	None
FreeMark [22]	CNN	Trusted third party + white-box access	Ownership verification	None
M-LOCK [97]	CNN	Owner secret + training access	Unauthorized inference	None
DeepIPR [32]	CNN	Owner secret + model weight access	Unauthorized inference, ownership verification	None
NNSplitter [145]	CNN	TEE-held restoration secret	Unauthorized inference	None
CoreLocker [125]	CNN	External key delivery	Unauthorized inference	None
NNLock [3]	CNN	External key delivery	Unauthorized inference	None
Phantom [6]	CNN	TEE	Unauthorized inference	None
ChaoW [68]	CNN/RNN	External key delivery	Unauthorized inference	Chaotic dynamics
TLG [67]	TF	TEE	Unauthorized inference	None
LoRO [129]	TF	TEE	Unauthorized inference	None
ChaosFormer	TF	PUF + restricted runtime	Unauthorized inference, ownership verification	Chaotic dynamics
ChaosFormer-S	TF	PUF + restricted runtime	Unauthorized inference, ownership verification	SPI

5.5 Controlled Layer-selection Study

To separate immediate accuracy degradation, disruption of hidden representations, and uncertainty about the encrypted layer identities, we conduct a controlled mechanism study using a fixed 256,233-image attacker sample (approximately 20% of the ImageNet training set). Across layer selections, we hold the encryption keys, data split, hyperparameters, augmentation and optimizer recipe, and random seeds fixed; the attacker uses AdamW with cosine scheduling, RandAugment, MixUp, CutMix, label smoothing, and FP16 training. We quantify structural degradation using CKA damage [59], defined as one minus the pre-retraining hidden-representation CKA averaged across transformer blocks, and quantify recovery using the normalized area under the validation-accuracy curve (Recovery AUC). Larger CKA damage indicates greater representation disruption, whereas lower final accuracy and Recovery AUC indicate stronger resistance.

Exhaustive enumeration. Under these frozen conditions, we enumerate all 1,419 ($C_{12}^4 + C_{12}^6$) possible $K = 4$ and $K = 6$ subsets of the 12-layer DeiT-S and ViT-B models and evaluate the validation accuracy of each subset. Regardless of the layout, every subset collapses the encrypted model to chance: the initial accuracy saturates at $\approx 0.1\%$, and the prediction agreement with the clean model is zero for all 1,419 subsets. This exhaustive result establishes that immediate accuracy collapse is layout-agnostic for $K \in \{4, 6\}$, while the saturation of initial accuracy shows that it cannot distinguish which layouts are more resistant to retraining; this motivates the representation- and recovery-based analyses that follow.

Preregistered five-epoch screen. We train 17 preregistered $K = 6$ DeiT-S selections—Top- K , Last- K , two clustered, three spread, and ten Random- K selections—for five epochs under the frozen protocol. As shown in Table 12, every selection starts near random accuracy, and initial accuracy does not predict recovery (Spearman $\rho = 0.132$, $p = 0.614$, $n = 17$); CKA damage, by contrast, strongly predicts lower recovery ($\rho = -0.689$, $p = 0.0022$), indicating that resistance reflects how thoroughly the encrypted layers corrupt the hidden representations consumed by later blocks.

Table 12. Five-epoch screening results for the nine reported rows from the 17 preregistered $K = 6$ DeiT-S selections.

Selection	CKA dmg.	Init. acc. (%)	5-epoch acc. (%)	5-epoch AUC (%)
Top- K	0.742	0.10	40.10	16.53
Last- K	0.355	0.10	61.32	44.31
Clustered-0	0.730	0.09	44.22	23.33
Clustered-1	0.460	0.09	56.91	37.16
Spread-0	0.675	0.11	54.70	32.09
Spread-1	0.535	0.11	65.72	52.97
Spread-2	0.660	0.07	58.44	40.60
Random-00	0.556	0.11	63.78	48.78
Random-01	0.678	0.11	45.96	23.51

Selection definitions: Layer indices are zero-based (0 to 11). Top- K greedily grows a six-layer set: at each round, it profiles every remaining layer when added to the current set and selects the candidate causing the largest accuracy drop. Last- K selects $\{6, 7, 8, 9, 10, 11\}$. Clustered selections contain consecutive layers: Clustered-0 is $\{0, 1, 2, 3, 4, 5\}$ and Clustered-1 is $\{3, 4, 5, 6, 7, 8\}$. Spread selections place one layer in each of six two-block depth intervals: Spread-0 is $\{0, 2, 4, 6, 8, 10\}$, Spread-1 is $\{1, 3, 5, 7, 9, 11\}$, and Spread-2 is $\{0, 3, 4, 7, 8, 11\}$. Random-00 and Random-01 are fixed preregistered draws $\{1, 2, 5, 7, 10, 11\}$ and $\{0, 2, 5, 6, 7, 8\}$, respectively. Numeric suffixes identify candidates rather than performance rankings. For brevity, we list only the two extremes among the ten random selections: Random-01 is the strongest and comes closest to Top- K , whereas Random-00 is the weakest and recovers most readily. These rows bracket the heterogeneity of Random- K ; the Spearman statistics in the text use all 17 selections.

Representative 20-epoch confirmation. We run the six representative selections in Table 13 for the full 20 epochs using three seeds on DeiT-S and one cross-model confirmation seed on ViT-B. CKA damage continues to predict Recovery AUC ($\rho = -0.829$, $p = 0.0416$, $n = 6$), and the Recovery-AUC ranking transfers closely from DeiT-S to ViT-B ($\rho = 0.943$, $p = 0.0048$).

Table 13. Controlled $K = 6$ retraining study.

Selection	DeiT CKA dmg.	DeiT final (%)	DeiT Rec. AUC (%)	ViT CKA dmg.	ViT final (%)	ViT Rec. AUC (%)
Top-K	0.742	61.36	45.80	0.738	65.35	49.44
Clustered-1	0.460	65.33	55.68	0.454	63.55	53.10
Spread-1	0.535	68.67	63.00	0.548	68.46	59.11
Random-00	0.556	68.27	61.31	0.538	68.78	58.57
Random-01	0.678	61.01	48.31	0.673	62.40	48.28
Random-02	0.649	64.07	53.43	0.672	64.19	50.93

The shared selection definitions follow Table 12; the additional Random-02 layout encrypts $\{0, 4, 5, 6, 7, 11\}$. DeiT-S values are means of three seeds, whose final-accuracy standard deviations are 0.02–0.10 percentage points. Clean top-1 accuracy is 77.80% for DeiT-S and 80.94% for ViT-B, so every attacked model remains 9–19 percentage points below its clean checkpoint after 20 epochs.

Oracle attacker. We give an oracle attacker the exact six encrypted layer identities, allow it to reinitialize those layers, and apply the same five-epoch training used in the preregistered screen. As shown in Table 14, the oracle improves five-epoch accuracy by only 3.09 percentage points on average (range 0.81–6.25). Layer-location uncertainty therefore remains secondary: most of the resistance persists after the identities are disclosed because the attacker must repair mutually dependent internal representations rather than merely locate the damaged parameters.

Table 14. Oracle-attacker evaluation on the six representative selections (DeiT-S, 3 seeds, identical frozen setup to the five-epoch screen). The oracle is told the encrypted layer identities and may reinitialize them before the identical five-epoch training; the last column is the improvement over the blind condition in percentage points.

Selection	Blind 5-epoch (%)	Oracle 5-epoch (%)	Δ (pp)
Top-K	40.10	46.35	6.25
Clustered-1	56.91	59.80	2.89
Spread-1	65.72	66.53	0.81
Random-00	63.78	64.77	0.99
Random-01	45.96	51.20	5.24
Random-02	54.03	56.38	2.35

Underlying mechanism. CKA damage measures how thoroughly the encrypted layers corrupt the hidden representations that downstream blocks consume; across the controlled study, it is the strongest predictor of recovery (Spearman $\rho = -0.689$ in the five-epoch screen and $\rho = -0.829$ in the 20-epoch confirmation). Top-K attains the highest CKA damage (0.742) but requires relatively expensive per-layer profiling at every greedy round to construct a deterministic selection path. A random selection can sample a configuration with comparably high CKA damage—Random-01 reaches 0.678 without any search—which explains why some random layouts are nearly as robust as Top-K without exposing a greedy selection path.

5.6 Ablation Study

5.6.1 Impact of Encrypting FFN. We perform an ablation study on the encryption of the FFN layer in the transformer layers. Table 15 reports the effectiveness of applying ACM encryption *only* to the attention weights. The experimental results show that significantly more layers must be encrypted to reach a comparable model degradation effect when the FFN is left untouched. As demonstrated in Sec. 5.3, encrypting more layers introduces additional overhead, while skipping the FFN does not measurably reduce inference latency. Therefore, our proposed method of encrypting both the attention and FFN weights is necessary to obtain a strong locking effect at minimal cost.

Table 15. Model accuracy (%) when encrypting only attention weights. The number in parentheses indicates the layer index. Encrypting only attention weights requires more layers to achieve satisfactory performance degradation.

Steps	ViT-B	ViT-B-384	ViT-B-8b	BeiT-B	DeiT-B	Swin-B
Acc_0 (%)	80.32	83.91	80.24	84.54	80.20	85.14
1	56.46 (11)	60.65 (11)	53.64 (11)	62.67 (1)	75.69 (10)	64.36 (11)
2	13.87 (10)	14.37 (10)	11.35 (10)	8.23 (2)	31.33 (11)	42.16 (10)
3	4.87 (9)	4.83 (9)	3.93 (9)	0.87 (3)	1.04 (9)	12.01 (9)
4	2.01 (8)	2.11 (8)	1.61 (8)	0.50 (6)	0.15 (8)	2.09 (8)
5	0.97 (7)	1.01 (0)	0.74 (7)	0.30 (8)	0.10 (7)	0.63 (7)
6	0.46 (0)	0.45 (7)	0.30 (0)	0.13 (5)	0.09 (5)	0.30 (0)
7	0.21 (6)	0.22 (6)	0.20 (6)	—	—	0.12 (1)
8	0.17 (5)	0.13 (5)	0.14 (5)	—	—	0.11 (6)

5.6.2 Impact of Extra Security Layers. To empirically validate our claim that adding extra security layers enhances robustness, we conduct a controlled experiment isolating this variable. We perform retraining attacks on ViT-B models obfuscated with a varying number of additional security layers, from zero to four. For each case, the attacker’s best attempt at recovering performance by retraining the model’s full parameters with 20% of the original training dataset is recorded.

The results, presented in Table 16, show a strong overall degradation trend, although it is not strictly monotonic because the added layers are randomly selected: accuracy decreases from 69.7% with no extra layer to 54.5% with three, then changes to 56.4% with four. The result is consistent with the selection heterogeneity observed in Table 13 and suggests that the benefit largely plateaus after three additional layers in this experiment. The CKA damage column of Table 16 mirrors this trend on the encrypted model itself: representation damage rises from 0.664 at the two-layer baseline to 0.751 at three extra layers, matching the drop in recovered accuracy from 69.7% to 54.5%, and the mild reversal at four extra layers (CKA 0.738, accuracy 56.4%) appears in both quantities. In practice, the number of extra layers can be chosen to balance this enhanced security against a minor increase in inference overhead.

6 Related Works

This section reviews IP protection methods for deep learning models. We hereby categorize previous efforts into **passive protection** and **proactive protection**, and within each category, we discuss the major sub-families and how ChaosFormer relates to them.

Table 16. Impact of extra security layers on retraining ViT-B. The baseline model required two layers to fall below the random-guess threshold; the “Number of Extra Security Layers” column refers to layers added on top of these initial two. Because the added layers are randomly selected, the trend need not be monotonic. CKA damage is one minus the hidden-representation linear CKA of the encrypted model before retraining, measured on 5,000 ImageNet validation samples under the same frozen encryption setup.

Number of Extra Security Layers	CKA damage	Best Top-1 Acc. after Retraining (%)
0	0.664	69.7
1	0.666	68.1
2	0.696	58.4
3	0.751	54.5
4	0.738	56.4

6.1 Passive Protection of DNN

Passive protection (sometimes called reactive protection) aims to enable model owners to *prove* prior possession after a theft has occurred, rather than to prevent the theft itself.

White-box watermarking. The earliest line of work assumes that the dispute resolver has access to the suspect model’s parameters. Uchida et al. [117] embed an owner-specific signature into the weights via an additional regularizer; DeepSigns [99] extends this to embedding into the activation distributions of selected layers; PCPT/ACPT [33] adds a copyright tracing layer that supports public verification. White-box methods provide clear ownership signals but require the verifier to physically access the disputed weights.

Black-box / trigger-set watermarking. A second line of work embeds a backdoor-style trigger set into the model: certain crafted inputs produce attacker-chosen outputs that the legitimate owner can use to claim ownership. Adi et al. [2] pioneered this construction; Zhang et al. [136] apply it to commercial DNN services; Liu et al. [70] embed the trigger as a greedy residual; Li et al. [64] design watermarks resistant to fine-tuning; DeepFidelity [47] studies the fidelity-robustness trade-off; explanation-based watermarks [102] use feature-attribution patterns. Recent work also explores training-free post-deployment watermarks [24, 49]. The fundamental limitation of black-box watermarking is that the verifier must be able to query the suspect model, which a sophisticated adversary running the stolen model privately can simply refuse.

Fingerprinting. Fingerprinting derives a model-specific identifier from the target model rather than embedding one. Sub-families include adversarial-example fingerprints [143], decision-boundary fingerprints [14], characteristic-example fingerprints [124], intrinsic fingerprints from edge-device behavior [90], hardware-bound non-repudiable fingerprints [144], recent robustness-based extensions [131], and FreeMark [22], a non-invasive white-box watermark operationally closer to keyed fingerprinting because it leaves the model unchanged. Verification requires either white-box access to the suspect model or its black-box query interface.

Robustness of passive schemes. A growing body of work demonstrates that passive schemes are fragile against adaptive adversaries. REFIT [21] shows watermark removal via fine-tuning under limited data; Shafieinejad et al. [101] attack backdoor watermarks; Sun et al. [107] remove watermarks via GANs; Yan et al. [132] and DeepEclipse [91] provide unified attack frameworks for breaking white-box watermarks; Pang et al. [89] and Jovanović et al. [56] show watermark stealing in LLMs. Lukas et al. [75] evaluate these schemes against a common adaptive adversary and find none of the surveyed methods robust in practice. Despite these limitations, watermarking and fingerprinting are still useful as a complement to the proactive protection we propose: ChaosFormer

prevents misuse on unauthorized devices, while a watermark or fingerprint can support post-hoc legal proceedings if a leak does occur.

6.2 Proactive Protection

Proactive schemes aim to block unauthorized inference from the outset, typically by requiring a correct key, license, or trusted hardware path. We organize them by mechanism.

Encryption-based methods. Several works lock neural networks with cryptographic keys without modifying the training process. NNLock [3] encrypts individual parameters with a lightweight scheme but incurs substantial CPU overhead. LEWIP [81] surveys recent DNN IP protection schemes and breaks several of them, motivating the need for formal security guarantees of the kind we provide via SPI. Goldstein et al. [39] build an obfuscation pipeline backed by a hardware security module. CoreLocker [125] and NNSplitter [145] identify a small “critical-weight” subset (often less than 1% of parameters) and protect only those weights; this minimal-perturbation strategy was shown by Sun et al. [108] to expose statistical fingerprints that an adaptive attacker can exploit, which is precisely the vulnerability that ChaosFormer (and especially ChaosFormer-S) is designed to avoid by encrypting whole layers and randomizing their distribution.

Model-level access control. A different family of proactive schemes embeds access control directly into the model’s behavior, granting full functionality only to users who possess specific secret information. Chen and Wu [20] add an active control module that disables inference under adversarial perturbations; DeepIPR [32] introduces passport layers whose scale and bias factors are correct only when a designated passport input is provided; M-LOCK [97] uses backdoor-style data poisoning for cybersecurity services; SecureNet [66] uses backdoor learning for proactive IP protection; AdvParams [130] encrypts parameters via adversarial perturbations; PSS performs probabilistic selective encryption for hierarchical services; EdgePro [18] authorizes individual neurons. The common drawback is that these schemes require retraining or fine-tuning with extra objectives, which becomes prohibitively expensive for transformer-scale models and lacks scalability when separate models must be issued per user.

PUF-based DNN protection. Using PUFs to bind applications to a specific device is a well-established idea in cryptographic hardware [13]. The use of PUFs specifically for DNN protection is more recent. Lin et al. [68] were the first to combine ACM-style chaotic permutation with hardware binding for CNNs and RNNs; Goldstein et al. [39] explore a related HSM-backed approach; PIPP [62] uses PUFs for FPGA-deployed DNN IP protection; Pan et al. [87] present a permute-diffusion scheme for MLPs and CNNs; Jiang et al. [55] add publicly verifiable ownership; Rajendran et al. [95] target binarized neural networks via PUF-based key management in memristive crossbars; Basak et al. [9] subsequently recover that scheme’s key bit-by-bit from an accuracy oracle, motivating our multi-layer partial-correctness experiment in Sec. 5.2.2; Guo et al. [41] present an early pay-per-device CNN IP-protection scheme. None of these works specifically address transformers — their encryption strategies, designed for CNN convolution kernels or MLP weight matrices, do not naturally fit attention layers (square matrices, quadratic in head dimension) or FFN layers (rectangular, $4\times$ embedding dimension). ChaosFormer is the first PUF-based scheme designed specifically for the transformer architecture, with separate primitives for attention (ACM) and FFN (Knuth shuffle) and formal SPI security in the diffusion-augmented variant.

TEE-based methods. TEEs protect model parameters during runtime by partitioning the inference into a trusted secure enclave and an untrusted accelerator. Representative on-device systems include DarknetTZ [79], which uses TrustZone for partial DNN inference. ShadowNet [109] and Occlumency [61] target on-device CNN inference; SOTER [103] guards black-box inference at the edge; MirrorNet [73] and ASGARD [80] extend TEE-based inference with virtualization. For transformers specifically, TransLinkGuard [67] secures layer permutations within a TEE but pays

for it with high inference latency; LoRO [129] uses low-rank obfuscation; Phantom [6] obfuscates the model architecture. DeepContract [146] uses TEEs to support remote license revocation. The fundamental issue with TEE-based schemes is the limited secure memory: 128 MB in Intel SGX1 and up to 16 MB for most ARM TrustZone devices, which is well below the size of modern transformer models, leading to memory swapping that dominates inference time [122, 142]. Side-channel work on TEEs [15, 23, 60, 134, 135, 139] has further demonstrated that the boundary between trusted and untrusted regions is itself an attack surface.

GPU-side TEE designs. Because the bulk of transformer computation happens on the GPU, several proposals extend TEE-style guarantees to the GPU. Graviton [120] requires hardware modifications to the GPU; HIX [53] extends SGX with heterogeneous device protection; GEVisor [127] and Honeycomb [77] provide GPU isolation through software adaptation. All of these approaches share a deployment cost: either the GPU silicon must change (typically a multi-year cycle) or substantial system-software work is required to adapt the application stack. ChaosFormer avoids those requirements: its software kernels require neither modified GPU silicon nor integration with a TEE system stack.

6.3 Stealing Attacks against On-device Models

Memory-extraction attacks. Several recent works demonstrate that runtime memory is a real attack surface for unprotected models. Sun et al. [110] survey insufficient model protection in mobile apps at scale; Deng et al. [26] characterize on-device DL model threats in Android; Oygenblik et al. [86] reconstruct full models from acquired CPU/GPU memory images; Ren et al. [98] identify model-stealing vulnerabilities in deployed mobile apps; Liu et al. [74] decompile DNN models from binaries; cold-boot attacks [40, 42, 82] and cache side channels [69, 96, 133] expose additional privileged or physical channels. ChaosFormer bounds simultaneous plaintext residency but does not fundamentally defeat a privileged and persistent runtime adversary. Crucially, this reliance on an uncompromised OS or execution environment is the standard trust boundary across contemporary model-locking literature: ChaoW decrypts and re-encrypts layers during local-device inference [68]; NNSplitter stores only sparse restoration secrets in a TEE and explicitly leaves TEE vulnerabilities outside its focus [145]; CoreLocker restores extracted weights through an externally secured key channel [125]; and the recent LymphNode preprint assumes signed firmware and excludes runtime-memory and intermediate-feature access [92]. EdgePro likewise acknowledges that temporarily decrypted authorization passwords remain a security issue [18]. Thus, ChaosFormer aligns with the established artifact-protection boundary of the field, protecting models at rest and in transit while requiring orthogonal hardware enforcement to defeat fully compromised execution stacks.

Architecture-stealing attacks. A separate line of work targets the model's architecture rather than its weights. DeepSniffer [46] infers architecture from execution traces; Zhang et al. [141] use FPGA side-channels; DeepTheft [37] steals architectures via power side-channels; NeurObfuscator [65] provides architectural obfuscation. ChaosFormer assumes the architecture is public (the standard assumption for transformer-family models), but as we note in Sec. 7, architectural obfuscation is a natural complement to weight encryption.

Our focus. Solutions above either focus on small networks (CNNs, MLPs), require retraining, depend on TEE primitives, or assume an attacker without partial training data and reference checkpoints. In contrast, the ChaosFormer framework encrypts transformer models using PUF-driven keys without retraining and avoids TEE constraints; its diffusion-augmented ChaosFormer-S variant provides the formal SPI guarantee and protection under the white-box plus public-checkpoint setting described in Sec. 3. To our knowledge, it is the first PUF-based active protection scheme designed specifically for transformers and the first to include an SPI-secure variant.

6.4 Limitations and Scope

Trust in the hardware vendor. Our protocol (Sec. 4.1) assumes the hardware vendor faithfully enrolls the PUF, protects the enrollment association ($DID, P, K_{\text{master}}$), and removes the unrestricted manufacturing and raw-response readout interface after registration. A malicious vendor could, in principle, leak K_{master} , issue unauthorized descendant keys, and clone device bindings. Ultimately, this allows the attacker to decrypt the obfuscated transformer models, leading to a total loss of IP protection; ChaosFormer supplies no security against a compromised cryptographic root authority. This manufacturer-rooted trust is not unique to our scheme. Prior work on SGX-based shielded execution explicitly identifies the processor manufacturer as the ultimate root of trust for attestation [10], while commercial secure OEM key-provisioning workflows rely on a trusted provisioning authority [83]. To mitigate this inherent supply-chain risk, vendors must enforce stringent industrial operational controls. Consistent with NIST organizational key-management guidance [8], these controls include storing K_{master} within Hardware Security Modules (HSMs), maintaining issuance inventories and audit logs, enforcing multi-party control across independent operators, conducting periodic compliance audits, and establishing compromise-recovery procedures. A PUF-specific control is post-enrollment verification that unrestricted raw-response access has been disabled.

PUF aging, reliability, and operational scope. While our evaluation demonstrates robust key reconstruction using an empirical SRAM PUF dataset and standard fuzzy extraction, this validation strictly confirms the algorithmic workflow rather than guaranteeing the lifelong physics of every fabricated device. Abideen et al. experimentally show that SRAM-macro orientation changes systematic startup-response bias patterns across otherwise comparable memories [1]. Because such bias must be accounted for when estimating usable entropy, raw response length alone does not establish the conditional min-entropy required for key extraction. Public datasets lack longitudinal aging traces, so predicting precise silicon degradation over decades remains outside the scope of this work. Consequently, practical commercial deployment requires per-layout and per-process qualification of conditional min-entropy and hardware drift before setting the extraction parameters and error-correction budget. Furthermore, our explicit “fail closed” security mechanism introduces a tangible operational limitation: the reliability budget of the chosen fuzzy extractor sets a hard cap on tolerable noise, and devices that drift beyond this cap due to extreme aging or environmental stress will halt decryption; in such cases, the user must re-enroll the device, which is a known operational cost of PUF-based systems.

Privileged runtime inspection. The prototype does not claim confidentiality once an adversary can debug or modify the authorized inference stack or read accelerator memory. Production application sandboxes and non-debuggable signed builds raise the privilege required for acquisition [4, 5], but a rooted device or compromised kernel/driver defeats this boundary. Layer-wise erasure only reduces exposure. Hardware-enforced confidential execution or a fused decrypt-compute accelerator is required when the device integrator itself is adversarial.

Language-model scope. Our LLM evaluation establishes scaling under dense FP16 prefill. Functional compatibility with standard contiguous INT8 weight storage is demonstrated by ViT-B-8b in Table 3; the diffusion implementation also operates directly on raw INT8 bit patterns (Sec. 4.2.3). Packed sub-byte formats such as INT4 and NF4 require packing-aware kernels, while sparse formats require format-specific handling to preserve their index or block structure and hardware acceleration. Autoregressive decoding introduces token-by-token execution and KV-cache management and therefore requires a distinct evaluation path. Our primary target is on-device edge inference: the 16.42 GB footprint measured for dense FP16 OPT-6.7B exceeds the 8 GB memory of our Jetson Orin Nano, while dense FP16 70B execution exceeds a single 24 GB RTX 4090 and therefore

requires server-class multi-GPU hardware. Accordingly, packed low-bit and sparse LLM execution, autoregressive decoding, and 70B-scale multi-GPU execution remain outside the evaluated scope. **Excluded attack classes.** As stated in Sec. 3, our threat model excludes invasive hardware attacks (decapping, focused-ion-beam probing) and direct side-channel attacks on the PUF circuitry itself (electromagnetic, photonic). Defending against these requires hardware-level countermeasures (shielding, sensor meshes, masking) that are orthogonal to the algorithmic protections we provide. The fixed-schedule result in Sec. 4.2.3 addresses only leakage of the secret ACM iteration count through execution timing; it does not establish resistance to cache, power, electromagnetic, PUF-circuit, or runtime-memory side channels. We also exclude indirect black-box model extraction through query-based substitute training [84, 85, 116], which requires a separate defense layer on the inference API [34, 63].

Architecture privacy. ChaosFormer protects the weights but assumes that the network architecture is public knowledge — a reasonable assumption for transformer-family models, where the architecture is essentially standardized. If architectural privacy is also required, ChaosFormer should be combined with an architectural-obfuscation layer [6, 65], which is one of the future directions discussed in our conclusion.

This manuscript substantially extends our preliminary work [48]. The primary new contributions include: (1) the introduction of ChaosFormer-S (Sec. 4.2), a diffusion-augmented variant ensuring that encrypted weights are computationally indistinguishable from uniform under the PRF assumption; (2) rigorous formal security proofs (e.g., Single-Point Indistinguishability) for the framework (Sec. 4.2.1); (3) extended robustness evaluations against domain-transfer attacks on complex datasets (OfficeHome, DomainNet) (Sec. 5.2); (4) detailed edge-deployment optimizations, including mitigation of iteration-count timing leakage and streaming GPU kernels (Sec. 4.2.3); (5) a hardware-bound functional ownership-evidence protocol with explicit statistical decision thresholds (Sec. 4.4), enabling public TTP verification and local verification without requiring access to the hardware vendor’s secure enrollment records; and (6) a substantially expanded discussion of related work, deployment scenarios, and limitations.

7 Conclusion

We propose an active, hardware-bound IP protection scheme that integrates PUF-based cryptographic key derivation, ACM-based weight obfuscation, and diffusion. Under the stated offline white-box artifact model, a copied encrypted model cannot be executed correctly without the authorized device and key. The base scheme targets the standard threat model, while ChaosFormer-S provides the SPI guarantee against the extended public-checkpoint adversary. The method requires no retraining and adds low measured runtime overhead.

Looking ahead, there are three clear paths to strengthen and scale this protection. First, while our current work focuses on Transformers, the core framework is adaptable to other architectures, such as CNNs. For 4D convolutional kernels, which are typically not square, one could apply a rectangular chaotic map such as the Baker’s Map [36] in place of ACM. Second, building on our 1.3B and 6.7B effectiveness and scaling results, we plan to optimize layer selection and key residency for autoregressive decoding, packed low-bit and sparse LLM execution, and 70B-scale multi-GPU execution. Third, while our method only protects the weights and assumes full knowledge of the architecture, we can further integrate architectural obfuscation [6, 65] to achieve deeper protection against architecture-stealing attacks [37, 46, 141]. By offering a low-cost, framework-agnostic lock on valuable model IP, our approach takes a practical step toward securing on-device deep learning, from edge applications to large-scale deployments.

Acknowledgments

This work was supported by the National Natural Science Foundation of China [grant number 62404192] and the Shenzhen Stability Science Program 2023.

References

- [1] Zain Ul Abideen, Rui Wang, Tiago Diadami Perez, Geert-Jan Schrijen, and Samuel Pagliarini. 2024. Impact of Orientation on the Bias of SRAM-Based PUFs. *IEEE Design & Test* 41, 3 (2024), 14–20. doi:10.1109/MDAT.2023.3322621
- [2] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. 2018. Turning Your Weakness Into a Strength: Watermarking Deep Neural Networks by Backdooring. In *Proc. 27th USENIX Secur. Symp.* USENIX Association, 1615–1631.
- [3] Manaar Alam, Sayandeep Saha, Debdeep Mukhopadhyay, and Sandip Kundu. 2022. NN-Lock: A Lightweight Authorization to Prevent IP Threats of Deep Learning Models. *ACM J. Emerg. Technol. Comput. Syst.* 18, 3 (2022), 51:1–51:19. doi:10.1145/3505634
- [4] Android Developers. 2024. android:debuggable. Android Privacy and Security Documentation. <https://developer.android.com/privacy-and-security/risks/android-debuggable> Accessed: 2026-07-12.
- [5] Android Open Source Project. 2026. Application Sandbox. Android Security Documentation. <https://source.android.com/docs/security/app-sandbox> Accessed: 2026-07-12.
- [6] Juyang Bai, Md Hafizul Islam Chowdhury, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. 2025. Phantom: Privacy-Preserving Deep Neural Network Model Obfuscation in Heterogeneous TEE and GPU System. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 5565–5582. <https://www.usenix.org/conference/usenixsecurity25/presentation/bai-juyang>
- [7] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. 2022. BEiT: BERT Pre-Training of Image Transformers. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=p-BhZSz59o4>
- [8] Elaine Barker and William Barker. 2019. *Recommendation for Key Management: Part 2–Best Practices for Key Management Organizations*. NIST Special Publication 800-57 Part 2 Rev. 1. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-57pt2r1
- [9] Bijeet Basak, Nupur Patil, Kurian Polachan, and Srinivas Vivek. 2026. Attack on a PUF-Based Secure Binary Neural Network. In *39th International Conference on VLSI Design and 25th International Conference on Embedded Systems (VLSID)*. 103–108. doi:10.1109/VLSID68508.2026.00032
- [10] Andrew Baumann, Marcus Peinado, and Galen Hunt. 2014. Shielding Applications from an Untrusted Cloud with Haven. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 267–283. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/baumann>
- [11] Mihir Bellare, Thomas Ristenpart, Phillip Rogaway, and Till Stegers. 2009. Format-Preserving Encryption. In *Selected Areas in Cryptography*, Vol. 5867. Springer, 295–312. doi:10.1007/978-3-642-05445-7_19
- [12] Daniel J Bernstein et al. 2008. ChaCha, a variant of Salsa20. In *Workshop record of SASC*, Vol. 8. Lausanne, Switzerland, 3–5.
- [13] Christoph Bösch, Jorge Guajardo, Ahmad-Reza Sadeghi, Jamshid Shokrollahi, and Pim Tuyls. 2008. Efficient Helper Data Key Extractor on FPGAs. In *Cryptogr. Hardware Embedded Syst. (CHES) (Lecture Notes in Computer Science, Vol. 5154)*. Springer, 181–197. doi:10.1007/978-3-540-85053-3_12
- [14] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. IPGuard: Protecting Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. In *ASIA CCS '21: ACM Asia Conference on Computer and Communications Security, Virtual Event, Hong Kong, June 7-11, 2021*, Jiannong Cao, Man Ho Au, Zhiqiang Lin, and Moti Yung (Eds.). ACM, 14–25. doi:10.1145/3433210.3437526
- [15] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the Prevailing Security Vulnerabilities in TrustZone-assisted TEE Systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 1416–1432. doi:10.1109/SP40000.2020.00061
- [16] Chip-Hong Chang, Yue Zheng, and Le Zhang. 2017. A retrospective and a look forward: Fifteen years of physical unclonable function advancement. *IEEE Circuits Syst. Mag.* 17, 3 (2017), 32–62. doi:10.1109/MCAS.2017.2713305
- [17] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.* 15, 3 (2024), 1–45.
- [18] Jinyin Chen, Haibin Zheng, Tao Liu, Jiawei Liu, Yao Cheng, Xuhong Zhang, and Shouling Ji. 2024. EdgePro: Edge Deep Learning Model Protection via Neuron Authorization. *IEEE Trans. Dependable Secur. Comput.* 21, 5 (2024), 4967–4981. doi:10.1109/TDSC.2024.3365730

- [19] Lei Chen and Shihong Wang. 2015. Differential Cryptanalysis of a Medical Image Cryptosystem with Multiple Rounds. *Computers in Biology and Medicine* 65 (2015), 69–75. doi:10.1016/j.combiomed.2015.07.024
- [20] Mingliang Chen and Min Wu. 2018. Protect Your Deep Neural Networks from Piracy. In *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*. IEEE, Hong Kong, China, 1–7. doi:10.1109/WIFS.2018.8630791
- [21] Xinyun Chen, Wenxiao Wang, Chris Bender, Yiming Ding, Ruoxi Jia, Bo Li, and Dawn Song. 2021. REFIT: A Unified Watermark Removal Framework For Deep Learning Systems With Limited Data. In *ACM Asia Conf. Comput. Commun. Secur. ACM*, 321–335. doi:10.1145/3433210.3453079
- [22] Yuzhang Chen, Jiangnan Zhu, Yujie Gu, Minoru Kuribayashi, and Kouichi Sakurai. 2024. FreeMark: A Non-Invasive White-Box Watermarking for Deep Neural Networks. *CoRR abs/2409.09996* (2024). doi:10.48550/arXiv.2409.09996
- [23] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. *IACR Cryptol. ePrint Arch.* 2016 (2016), 86. <http://eprint.iacr.org/2016/086>
- [24] Kieu Dang, Phung Lai, NhatHai Phan, Yelong Shen, Ruoming Jin, Abdallah Khreishah, and My Thai. 2025. SoK: Are Watermarks in LLMs Ready for Deployment? *ArXiv preprint abs/2506.05594* (2025). <https://arxiv.org/abs/2506.05594>
- [25] Jeroen Delvaux, Dawu Gu, Ingrid Verbauwhede, Matthias Hiller, and Meng-Day Yu. 2016. Efficient fuzzy extraction of PUF-induced secrets: Theory and applications. In *International Conference on Cryptographic Hardware and Embedded Systems*. Springer, 412–431.
- [26] Zizhuang Deng, Kai Chen, Guozhu Meng, Xiaodong Zhang, Ke Xu, and Yao Cheng. 2022. Understanding Real-world Threats to Deep Learning Models in Android Apps. In *ACM SIGSAC Conf. Comput. Commun. Secur. ACM*, 785–799. doi:10.1145/3548606.3559388
- [27] Giuseppe Desolda, Lauren S. Ferro, Andrea Marrella, Tiziana Catarci, and Maria Francesca Costabile. 2021. Human Factors in Phishing Attacks: A Systematic Literature Review. *ACM Comput. Surv.* 54, 8, Article 173 (2021), 35 pages. doi:10.1145/3469886
- [28] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. 2008. Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data. *SIAM J. Comput.* 38, 1 (2008), 97–139. doi:10.1137/060651380
- [29] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *Proc. of ICLR*. OpenReview.net. <https://openreview.net/forum?id=YicbFdNTTy>
- [30] Abdulrahman Abu Elkhail, Anys Bacha, and Hafiz Malik. 2025. Sniper: Countering Locker Ransomware Attacks Through Natural Language Processing. *IEEE Trans. Dependable Secur. Comput.* 22, 4 (2025), 4160–4175. doi:10.1109/TDSC.2025.3543268
- [31] Mahmoud F. Abd Elzaher, Mohamed Shalaby, and Salwa H. El Ramly. 2016. An Arnold Cat Map-Based Chaotic Approach for Securing Voice Communication. In *Proceedings of the 10th International Conference on Informatics and Systems*. ACM, Giza Egypt, 329–331. doi:10.1145/2908446.2908508
- [32] Lixin Fan, Kam Woh Ng, Chee Seng Chan, and Qiang Yang. 2022. DeepIPR: Deep Neural Network Ownership Verification With Passports. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 10 (2022), 6122–6139. doi:10.1109/TPAMI.2021.3088846
- [33] Xuefeng Fan, Dahao Fu, Hangyu Gui, Xinpeng Zhang, and Xiaoyi Zhou. 2022. PCPT and ACPT: Copyright Protection and Traceability Scheme for DNN Models. <https://arxiv.org/abs/2206.02541>
- [34] Ryan Feng, Ashish Hooda, Neal Mangaokar, Kassem Fawaz, Somesh Jha, and Atul Prakash. 2023. Stateful Defenses for Machine Learning Models Are Not Yet Secure Against Black-box Attacks. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM, 786–800. doi:10.1145/3576915.3623116
- [35] Jiri Fridrich. 1998. Symmetric Ciphers Based on Two-Dimensional Chaotic Maps. *Int. J. Bifurcation and Chaos* 08, 06 (1998), 1259–1284. doi:10.1142/S021812749800098X
- [36] Jiri Fridrich. 1998. Symmetric ciphers based on two-dimensional chaotic maps. *Int. J. Bifurcat. Chaos* 8, 06 (1998), 1259–1284.
- [37] Yansong Gao, Huming Qiu, Zhi Zhang, Binghui Wang, Hua Ma, Alsharif Abuadbbba, Minhui Xue, Anmin Fu, and Surya Nepal. 2024. DeepTheft: Stealing DNN Model Architectures through Power Side Channel. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 3311–3326. doi:10.1109/SP54263.2024.00250
- [38] Blaise Gassend, Marten van Dijk, Dwaine E. Clarke, Emina Torlak, Srinivas Devadas, and Pim Tuyls. 2008. Controlled physical random functions and applications. *ACM Trans. Inf. Syst. Secur.* 10, 4 (2008), 3:1–3:22. doi:10.1145/1284680.1284683
- [39] Brunno F. Goldstein, Vinay C. Patil, Victor da Cruz Ferreira, Alexandre Solon Nery, Felipe M. G. França, and Sandip Kundu. 2021. Preventing DNN Model IP Theft via Hardware Obfuscation. *IEEE J. Emerg. Sel. Topics Circuits Syst.* 11, 2 (2021), 267–277. doi:10.1109/JETCAS.2021.3076151

- [40] Michael Gruhn and Tilo Müller. 2013. On the Practicability of Cold Boot Attacks. In *2013 International Conference on Availability, Reliability and Security, ARES 2013, Regensburg, Germany, September 2-6, 2013*. IEEE Computer Society, 390–397. doi:10.1109/ARES.2013.52
- [41] Qingli Guo, Jing Ye, Yue Gong, Yu Hu, and Xiaowei Li. 2018. PUF Based Pay-Per-Device Scheme for IP Protection of CNN Model. In *Proc. 2018 IEEE 27th Asian Test Symposium (ATS)*. IEEE, Hefei, China, 115–120. doi:10.1109/ATS.2018.00032 Presented at the 2018 IEEE 27th Asian Test Symposium (ATS), October 15–18, Hefei, China.
- [42] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. 2009. Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM* 52, 5 (2009), 91–98. doi:10.1145/1506409.1506429
- [43] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. 2022. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 1 (2022), 87–110.
- [44] Grant Ho, Asaf Cidon, Lior Gavish, Marco Schweighauser, Vern Paxson, Stefan Savage, Geoffrey M. Voelker, and David A. Wagner. 2019. Detecting and Characterizing Lateral Phishing at Scale. In *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, Nadia Heninger and Patrick Traynor (Eds.). USENIX Association, 1273–1290. <https://www.usenix.org/conference/usenixsecurity19/presentation/ho>
- [45] Daniel E. Holcomb, Wayne P. Burleson, and Kevin Fu. 2009. Power-Up SRAM State as an Identifying Fingerprint and Source of True Random Numbers. *IEEE Trans. Computers* 58, 9 (2009), 1198–1210. doi:10.1109/TC.2008.212
- [46] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, et al. 2020. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proc. 25th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*. 385–399.
- [47] Guang Hua and Andrew Beng Jin Teoh. 2023. Deep fidelity in DNN watermarking: A study of backdoor watermarking for classification models. *Pattern Recognit.* 144 (2023), 109844. doi:10.1016/J.PATCOG.2023.109844
- [48] Peichun Hua, Hanxiu Zhang, Tuo Li, and Yue Zheng. 2025. Securing On-device Transformer with Hardware Binding and Reversible Obfuscation. In *IEEE Annual Computer Security Applications Conference, ACSAC 2025, Honolulu, HI, USA, December 8-12, 2025*. IEEE, 956–972. doi:10.1109/ACSAC67867.2025.00079
- [49] Yujin Huang, Zhi Zhang, Qingchuan Zhao, Xingliang Yuan, and Chunyang Chen. 2025. THEMIS: Towards Practical Intellectual Property Protection for Post-Deployment On-Device Deep Learning Models. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 7311–7330. <https://www.usenix.org/conference/usenixsecurity25/presentation/huang-yujin>
- [50] Yujin Huang, Xin Zheng, Xingliang Yuan, and Kwok-Yan Lam. 2026. SoK: Attack and Defense Landscape of Mobile On-device AI Systems. *CoRR abs/2607.00362* (2026). arXiv:2607.00362 doi:10.48550/ARXIV.2607.00362
- [51] Usman Inayat, Mashaim Farzan, Sajid Mahmood, Muhammad Fahad Zia, Shahid Hussain, and Fabiano Pallonetto. 2024. Insider threat mitigation: Systematic literature review. *Ain Shams Engineering Journal* 15, 12 (2024), 103068.
- [52] Intrinsic ID. 2021. *SRAM PUF: The Secure Silicon Fingerprint*. White Paper. Intrinsic ID. <https://data.embeddedcomputing.com/uploads/articles/whitepapers/14491.pdf> Accessed: July 18, 2026.
- [53] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. 2019. Heterogeneous isolated execution for commodity gpus. In *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*. 455–468.
- [54] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *ArXiv preprint abs/2406.00515* (2024). <https://arxiv.org/abs/2406.00515>
- [55] Jingdong Jiang, Yue Zheng, and Chip-Hong Chang. 2025. PUF-based Edge DNN Model IP Protection with Self-obfuscation and Publicly Verifiable Ownership. In *IEEE International Symposium on Circuits and Systems, ISCAS 2025, London, United Kingdom, May 25-28, 2025*. IEEE, 1–5. doi:10.1109/ISCAS56072.2025.11044032
- [56] Nikola Jovanovic, Robin Staab, and Martin T. Vechev. 2024. Watermark Stealing in Large Language Models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=Wp054bnPq9>
- [57] Hyunho Kang, Yohei Hori, Toshihiro Katashita, Manabu Hagiwara, and Keiichi Iwamura. 2014. Cryptographic key generation from PUF data using efficient fuzzy extractors. In *16th International conference on advanced communication technology*. IEEE, 23–26.
- [58] Jeremie S. Kim, Minesh Patel, Hasan Hassan, and Onur Mutlu. 2018. The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency-Reliability Tradeoff in Modern Commodity DRAM Devices. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Washington, DC, USA, 194–207. doi:10.1109/HPCA.2018.00026
- [59] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. 2019. Similarity of Neural Network Representations Revisited. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 3519–3529. <https://proceedings.mlr.press/v97/kornblith19a.html>
- [60] Zili Kou, Sharad Sinha, Wenjian He, and Wei Zhang. 2023. Cache Side-channel Attacks and Defenses of the Sliding Window Algorithm in TEEs. In *Proc. 2023 Design, Automat. Test Europe Conf. Exhib. (DATE)*. IEEE, Antwerp, Belgium,

- 1–6. doi:10.23919/DATE56975.2023.10137116
- [61] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and Junehwa Song. 2019. Occlumency: Privacy-preserving remote deep-learning inference using SGX. In *The 25th Annual international conference on mobile computing and networking*. 1–17.
- [62] Dawei Li, Yangkun Ren, Di Liu, Yuxiao Guo, Zhenyu Guan, and Jianwei Liu. 2024. PIPP: A Practical PUF-Based Intellectual Property Protection Scheme for DNN Model on FPGA. *IEEE Trans. Circuits Syst. II Express Briefs* 71, 2 (2024), 912–916. doi:10.1109/TCSII.2023.3309105
- [63] Huiying Li, Shawn Shan, Emily Wenger, Jiayun Zhang, Haitao Zheng, and Ben Y. Zhao. 2022. Blacklight: Scalable Defense for Neural Networks against Query-Based Black-Box Attacks. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2117–2134. <https://www.usenix.org/conference/usenixsecurity22/presentation/li-huiying>
- [64] Huiying Li, Emily Wenger, Shawn Shan, Ben Y. Zhao, and Haitao Zheng. 2019. Piracy Resistant Watermarks for Deep Neural Networks. <https://arxiv.org/abs/1910.01226>
- [65] Jingtao Li, Zhezhi He, Adnan Siraj Rakin, Deliang Fan, and Chaitali Chakrabarti. 2021. Neurobfuscatore: A full-stack obfuscation tool to mitigate neural architecture stealing. In *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*. IEEE, 248–258.
- [66] Peihao Li, Jie Huang, Huaqing Wu, Zeping Zhang, and Chunyang Qi. 2024. SecureNet: Proactive intellectual property protection and model security defense for DNNs based on backdoor learning. *Neural Netw.* 174 (2024), 106199. doi:10.1016/J.NEUNET.2024.106199
- [67] Qinfeng Li, Zhiqiang Shen, Zhenghan Qin, Yangfan Xie, Xuhong Zhang, Tianyu Du, Sheng Cheng, Xun Wang, and Jianwei Yin. 2024. TransLinkGuard: Safeguarding Transformer Models Against Model Stealing in Edge Deployment. In *Proc. 32nd ACM Int. Conf. Multimedia*. ACM, 3479–3488. doi:10.1145/3664647.3680786
- [68] Ning Lin, Xiaoming Chen, Hang Lu, and Xiaowei Li. 2021. Chaotic Weights: A Novel Approach to Protect Intellectual Property of Deep Neural Networks. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 40, 7 (2021), 1327–1339. doi:10.1109/TCAD.2020.3018403
- [69] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. 2015. Last-Level Cache Side-Channel Attacks are Practical. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, 605–622. doi:10.1109/SP.2015.43
- [70] Hanwen Liu, Zhenyu Weng, and Yuesheng Zhu. 2021. Watermarking Deep Neural Networks with Greedy Residuals. In *Proc. of ICML (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 6978–6988. <http://proceedings.mlr.press/v139/liu21x.html>
- [71] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).
- [72] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 9992–10002. doi:10.1109/ICCV48922.2021.00986
- [73] Ziyu Liu, Yukui Luo, Shijin Duan, Tong Zhou, and Xiaolin Xu. 2023. Mirrornet: A tee-friendly framework for secure on-device dnn inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [74] Zhibo Liu, Yuanyuan Yuan, Shuai Wang, Xiaofei Xie, and Lei Ma. 2023. Decompiling x86 deep neural network executables. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 2023)*. USENIX Association, Anaheim, CA, USA, 7357–7374. <https://www.usenix.org/conference/usenixsecurity23/presentation/liu-zhibo>
- [75] Nils Lukas, Edward Jiang, Xinda Li, and Florian Kerschbaum. 2022. SoK: How Robust is Image Classification Deep Neural Network Watermarking?. In *IEEE Symposium on Security and Privacy*. 787–804. doi:10.1109/SP46214.2022.9833693
- [76] Roel Maes and Vincent van der Leest. 2014. Countering the Effects of Silicon Aging on SRAM PUFs. In *Proceedings of the 2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 148–153. doi:10.1109/HST.2014.6855586
- [77] Haohui Mai, Jiacheng Zhao, Hongren Zheng, Yiyang Zhao, Zibin Liu, Mingyu Gao, Cong Wang, Huimin Cui, Xiaobing Feng, and Christos Kozyrakis. 2023. Honeycomb: Secure and efficient {GPU} executions via static validation. In *Proc. 17th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*. 155–172.
- [78] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*.
- [79] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. Darknetz: towards model privacy at the edge using trusted execution environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*. 161–174.

- [80] Myungsuk Moon, Minhee Kim, Joonkyo Jung, and Dokyung Song. 2025. ASGAR: Protecting On-Device Deep Neural Networks with Virtualization-Based Trusted Execution Environments. In *Proceedings 2025 Network and Distributed System Security Symposium*.
- [81] Rijoy Mukherjee and Rajat Subhra Chakraborty. 2023. Attacks on Recent DNN IP Protection Techniques and Their Mitigation. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 42, 11 (2023), 3642–3650. doi:10.1109/TCAD.2023.3272271
- [82] Tilo Müller and Michael Spreitzenbarth. 2013. FROST. In *Applied Cryptography and Network Security*, Michael Jacobson, Michael Locasto, Payman Mohassel, and Reihaneh Safavi-Naini (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 373–388.
- [83] NXP Semiconductors. 2025. *Secure OEM Key Provisioning with EdgeLock Secure Enclave on RT1180*. Application Note AN14861. NXP Semiconductors. <https://docs.nxp.com/bundle/AN14861/page/topics/introduction.html> Rev. 1.0, 13 November 2025; accessed 30 July 2026.
- [84] Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. 2023. I Know What You Trained Last Summer: A Survey on Stealing Machine Learning Models and Defences. *Comput. Surveys* 55, 14s (2023), 1–41. doi:10.1145/3595292
- [85] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff Nets: Stealing Functionality of Black-Box Models. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 4954–4963. doi:10.1109/CVPR.2019.00509
- [86] David Oygenblik, Carter Yagemann, Joseph Zhang, Arianna Mastali, Jeman Park, and Brendan Saltaformaggio. 2024. AI Psychiatry: Forensic Investigation of Deep Learning Networks in Memory Images. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, USA. <https://www.usenix.org/conference/usenixsecurity24/presentation/oygenblik>
- [87] Qianqian Pan, Mianxiong Dong, Kaoru Ota, and Jun Wu. 2022. Device-Bind Key-Storageless Hardware AI Model IP Protection: A PUF and Permuted-Diffusion Encryption-Enabled Approach. <https://arxiv.org/abs/2212.11133>
- [88] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. 2015. LibriSpeech: An ASR Corpus Based on Public Domain Audio Books. In *IEEE International Conference on Acoustics, Speech and Signal Processing*. 5206–5210. doi:10.1109/ICASSP.2015.7178964
- [89] Qi Pang, Shengyuan Hu, Wenting Zheng, and Virginia Smith. 2024. Attacking llm watermarks by exploiting their strengths. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.
- [90] Kartik Patwari, Syed Mahbub Hafiz, Han Wang, Houman Homayoun, Zubair Shafiq, and Chen-Nee Chuah. 2022. DNN model architecture fingerprinting attack on CPU-GPU edge devices. In *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 337–355.
- [91] Alessandro Pegoraro, Carlotta Segna, Kavita Kumari, and Ahmad-Reza Sadeghi. 2024. DeepEclipse: How to Break White-Box DNN-Watermarking Schemes. In *Proc. 33rd USENIX Secur. Symp.* USENIX Association.
- [92] Hanyu Pei, Shang Liu, and Zeyan Liu. 2026. LymphNode: A Plug-and-Play Access Control Method for Deep Neural Networks. *CoRR* abs/2605.16227 (2026). doi:10.48550/arXiv.2605.16227
- [93] Kuheli Pratihari, Soumi Chatterjee, Rajat Subhra Chakraborty, and Debdeep Mukhopadhyay. 2024. Enhancing SRAM-Based PUF Reliability Through Machine Learning-Aided Calibration Techniques. *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.* 43, 11 (2024), 3491–3502. doi:10.1109/TCAD.2024.3449570
- [94] Mahmood Azhar Qureshi and Arslan Munir. 2022. PUF-RAKE: A PUF-Based Robust and Lightweight Authentication and Key Establishment Protocol. *IEEE Trans. Dependable Secure Comput.* 19, 4 (2022), 2457–2475. doi:10.1109/TDSC.2021.3059454
- [95] Gokulnath Rajendran, Debajit Basak, Suman Deb, and Anupam Chattopadhyay. 2024. Securing binarized neural networks via PUF-based key management in memristive crossbar arrays. *IEEE Embed. Syst. Lett.* 17, 1 (2024), 30–33.
- [96] Adnan Siraj Rakin, Md Hafizul Islam Chowdhury, Fan Yao, and Deliang Fan. 2022. DeepSteal: Advanced Model Extractions Leveraging Efficient Weight Stealing in Memories. In *IEEE Symp. Secur. Privacy (SP)*. IEEE, San Francisco, CA, USA, 1157–1174.
- [97] Ge Ren, Jun Wu, Gaolei Li, Shenghong Li, and Mohsen Guizani. 2024. Protecting Intellectual Property With Reliable Availability of Learning Models in AI-Based Cybersecurity Services. *IEEE Trans. Dependable Secur. Comput.* 21, 2 (2024), 600–617. doi:10.1109/TDSC.2022.3222972
- [98] Pengcheng Ren, Chaoshun Zuo, Xiaofeng Liu, Wenrui Diao, Qingchuan Zhao, and Shanqing Guo. 2024. Demistify: Identifying on-device machine learning models stealing and reuse vulnerabilities in mobile apps. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [99] Bita Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. 2019. DeepSigns: An End-to-End Watermarking Framework for Ownership Protection of Deep Neural Networks. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck (Eds.). ACM, 485–497. doi:10.1145/3297858.3304051

- [100] Abhishek Saini, Haolin Jiang, and Hang Liu. 2026. Vulnerabilities in Partial TEE-Shielded LLM Inference with Precomputed Noise. *ArXiv preprint* abs/2602.11088 (2026). <https://arxiv.org/abs/2602.11088>
- [101] Masoumeh Shafieinejad, Nils Lukas, Jiaqi Wang, Xinda Li, and Florian Kerschbaum. 2021. On the Robustness of Backdoor-based Watermarking in Deep Neural Networks. In *IH&MMSec '21: ACM Workshop on Information Hiding and Multimedia Security*, Dirk Borghys, Patrick Bas, Luisa Verdoliva, Tomás Pevný, Bin Li, and Jennifer Newman (Eds.). ACM, 177–188. doi:10.1145/3437880.3460401
- [102] Shuo Shao, Yiming Li, Hongwei Yao, Yiling He, Zhan Qin, and Kui Ren. 2024. Explanation as a watermark: Towards harmless and multi-bit model ownership verification via watermarking feature attribution. *ArXiv preprint* abs/2405.04825 (2024). <https://arxiv.org/abs/2405.04825>
- [103] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xiapu Luo, et al. 2022. {SOTER}: Guarding black-box inference for general neural networks at the edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 723–738.
- [104] Victor Shoup. 2004. Sequences of games: a tool for taming complexity in security proofs. *cryptology eprint archive* (2004).
- [105] Hemavathy Sriramulu and V. S. Kanchana Bhaaskaran. 2023. Arbiter PUF - A Review of Design, Composition, and Security Aspects. *IEEE Access* 11 (2023), 33979–34004. doi:10.1109/ACCESS.2023.3264016
- [106] G. Edward Suh and Srinivas Devasdas. 2007. Physical Unclonable Functions for Device Authentication and Secret Key Generation. In *Proc. 44th Design Automat. Conf. (DAC)*. IEEE, 9–14. doi:10.1145/1278480.1278484
- [107] Shichang Sun, Haoqi Wang, Mingfu Xue, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2021. Detect and Remove Watermark in Deep Neural Networks via Generative Adversarial Networks. In *Information Security - 24th International Conference, ISC 2021, Virtual Event, November 10-12, 2021, Proceedings (Lecture Notes in Computer Science, Vol. 13118)*, Joseph K. Liu, Sokratis K. Katsikas, Weizhi Meng, Willy Susilo, and Rolly Intan (Eds.). Springer, 341–357. doi:10.1007/978-3-030-91356-4_18
- [108] Yulian Sun, Vedant Bonde, Li Duan, and Yong Li. 2025. Obfuscation for Deep Neural Networks Against Model Extraction: Attack Taxonomy and Defense Optimization. In *Int. Conf. Appl. Cryptogr. Netw. Secur. (ACNS) (Lecture Notes in Computer Science, Vol. 15827)*, Marc Fischlin and Veelasha Moonsamy (Eds.). Springer, 391–414. doi:10.1007/978-3-031-95767-3_15
- [109] Zhichuang Sun, Ruimin Sun, Changming Liu, Amrita Roy Chowdhury, Long Lu, and Somesh Jha. 2023. ShadowNet: A Secure and Efficient On-device Model Inference System for Convolutional Neural Networks. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 1596–1612. doi:10.1109/SP46215.2023.10179382
- [110] Zhichuang Sun, Ruimin Sun, Long Lu, and Alan Mislove. 2021. Mind Your Weight(s): A Large-scale Study on Insufficient Machine Learning Model Protection in Mobile Apps. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, Michael D. Bailey and Rachel Greenstadt (Eds.). USENIX Association, 1955–1972. <https://www.usenix.org/conference/usenixsecurity21/presentation/sun-zhichuang>
- [111] Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proc. of ICLR (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 6105–6114. <http://proceedings.mlr.press/v97/tan19a.html>
- [112] Fatemeh Tehranipoor, Nima Karimian, Wei Yan, and John A. Chandy. 2017. DRAM-Based Intrinsic Physically Unclonable Functions for System-Level Security and Authentication. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 3 (2017), 1085–1097. doi:10.1109/TVLSI.2016.2606658
- [113] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, Tim Mattson, Abdullah Muzahid, and Armando Solar-Lezama (Eds.). ACM, 10–19. doi:10.1145/3315508.3329973
- [114] Hakan Tora, Erhan Gokcay, Mehmet Turan, and Mohamed Buker. 2022. A generalized Arnold's Cat Map transformation for image scrambling. *Multimedia Tools and Applications* 81, 22 (2022), 31349–31362. doi:10.1007/s11042-022-11985-2
- [115] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research)*, Marina Meila and Tong Zhang (Eds.). PMLR, 10347–10357. <http://proceedings.mlr.press/v139/touvron21a.html>
- [116] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs. In *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, Austin, TX, 601–618. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/tramer>
- [117] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding Watermarks into Deep Neural Networks. In *ACM Int. Conf. Multimedia Retrieval (ICMR)*. ACM, 269–277. doi:10.1145/3078971.3078974

- [118] Marten van Dijk and Chenglu Jin. 2023. A Theoretical Framework for the Analysis of Physical Unclonable Function Interfaces and Its Relation to the Random Oracle Model. *Journal of Cryptology* 36, 4 (2023), 1–64. doi:10.1007/s00145-023-09475-1
- [119] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [120] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on {GPUs}. In *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*. 681–696.
- [121] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *Proc. 2018 EMNLP Workshop BlackboxNLP*. 353–355.
- [122] Jinwen Wang, Yuhao Wu, Han Liu, Bo Yuan, Roger D. Chamberlain, and Ning Zhang. 2023. IP Protection in TinyML. In *60th ACM/IEEE Design Automation Conference, DAC 2023, San Francisco, CA, USA, July 9-13, 2023*. IEEE, 1–6. doi:10.1109/DAC56929.2023.10247898
- [123] Pengli Wang, Bingyou Dong, Yifeng Cai, Zheng Zhang, Junlin Liu, Huanran Xue, Ye Wu, Yao Zhang, and Ziqi Zhang. 2025. Game of Arrows: On the (In-)Security of Weight Obfuscation for On-Device TEE-Shielded LLM Partition Algorithms. In *Proc. 34th USENIX Secur. Symp.* USENIX Association, 279–298.
- [124] Siyue Wang, Xiao Wang, Pin-Yu Chen, Pu Zhao, and Xue Lin. 2021. Characteristic Examples: High-Robustness, Low-Transferability Fingerprinting of Neural Networks. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021*, Zhi-Hua Zhou (Ed.). ijcai.org, 575–582. doi:10.24963/IJCAI.2021/80
- [125] Zihan Wang, Zhongkui Ma, Xinguo Feng, Ruoxi Sun, Hu Wang, Minhui Xue, and Guangdong Bai. 2024. CORELOCKER: Neuron-level Usage Control. In *IEEE Symp. Secur. Privacy (SP)*. IEEE, 2497–2514. doi:10.1109/SP54263.2024.00233
- [126] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2023. Transformers in Time Series: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 6778–6786. doi:10.24963/IJCAI.2023/759
- [127] Xiaolong Wu, Dave (Jing) Tian, and Chung Hwan Kim. 2023. Building GPU TEEs using CPU Secure Enclaves with GEVisor. In *Proc. 14th ACM Symp. Cloud Comput. (SOCC)*. Santa Cruz, CA. doi:10.1145/3620678.3624659
- [128] Yue Wu, Zhongyun Hua, and Yicong Zhou. 2015. n -dimensional discrete cat map generation using Laplace Expansions. *IEEE transactions on cybernetics* 46, 11 (2015), 2622–2633.
- [129] Gaojian Xiong, Yu Sun, Jianhua Liu, Jian Cui, and Jianwei Liu. 2025. LoRO: Real-Time on-Device Secure Inference for LLMs via TEE-Based Low Rank Obfuscation. In *39th Annu. Conf. Neural Inf. Process. Syst.*
- [130] Mingfu Xue, Zhiyu Wu, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2023. AdvParams: An Active DNN Intellectual Property Protection Technique via Adversarial Perturbation Based Parameter Encryption. *IEEE Trans. Emerg. Top. Comput.* 11, 3 (2023), 664–678. doi:10.1109/TETC.2022.3231012
- [131] Anli Yan, Huali Ren, Kanghua Mo, Zhenxin Zhang, Shaowei Wang, and Jin Li. 2025. Enhancing Model Intellectual Property Protection With Robustness Fingerprint Technology. *IEEE Trans. Inf. Forensics Secur.* 20 (2025), 9235–9249. doi:10.1109/TIFS.2025.3602244
- [132] Yifan Yan, Xudong Pan, Mi Zhang, and Min Yang. 2023. Rethinking White-Box Watermarks on Deep Learning Models under Neural Structural Obfuscation. In *Proc. 32nd USENIX Secur. Symp.* USENIX Association, 2347–2364.
- [133] Yuval Yarom and Katrina Falkner. 2014. Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. In *23rd USENIX Security Symposium (USENIX Security 14)*. USENIX Association, San Diego, CA, 719–732. <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom>
- [134] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2024. CipherSteal: Stealing Input Data from TEE-Shielded Neural Networks with Ciphertext Side Channels. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, San Francisco, CA, USA, 79–79. <https://www.ieee-security.org/TC/SP2025/>
- [135] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2024. Hypertheft: Thieving model weights from TEE-shielded neural networks via ciphertext side channels. In *Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*. 4346–4360.
- [136] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. 2018. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security*. 159–172.
- [137] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4 (2025), 1–28.

- [138] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myale Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).
- [139] Xiaohan Zhang, Jinwen Wang, Yueqiang Cheng, Qi Li, Kun Sun, Yao Zheng, Ning Zhang, and Xinghua Li. 2024. Interface-Based Side Channel in TEE-Assisted Networked Services. *IEEE/ACM Transactions on Networking* 32, 1 (2024), 613–626. doi:10.1109/TNET.2023.3294019
- [140] Yinxing Zhang, Zhongyun Hua, Han Bao, Hejiao Huang, and Yicong Zhou. 2022. An n -Dimensional Chaotic System Generation Method Using Parametric Pascal Matrix. *IEEE Trans. Ind. Informatics* 18, 12 (2022), 8434–8444. doi:10.1109/TII.2022.3151984
- [141] Yicheng Zhang, Rozhin Yasaei, Hao Chen, Zhou Li, and Mohammad Abdullah Al Faruque. 2021. Stealing neural network structure through remote FPGA side-channel analysis. *IEEE Trans. Inf. Forensics Secur.* 16 (2021), 4377–4388.
- [142] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. 2024. No Privacy Left Outside: On the (In-)Security of TEE-Shielded DNN Partition for On-Device ML. In *2024 IEEE Symp. Secur. Privacy (SP)*. IEEE Computer Society, Washington, DC, USA, 3327–3345. doi:10.1109/SP54263.2024.00052
- [143] Jingjing Zhao, Qingyue Hu, Gaoyang Liu, Xiaoqiang Ma, Fei Chen, and Mohammad Mehedi Hassan. 2020. AFA: Adversarial fingerprinting authentication for deep neural networks. *Computer Communications* 150 (2020), 488–497.
- [144] Yue Zheng, Si Wang, and Chip-Hong Chang. 2022. A DNN Fingerprint for Non-Repudiable Model Ownership Identification and Piracy Detection. *IEEE Trans. Inf. Forensics Secur.* 17 (2022), 2977–2989. doi:10.1109/TIFS.2022.3198267
- [145] Tong Zhou, Yukui Luo, Shaolei Ren, and Xiaolin Xu. 2023. NNSplitter: An Active Defense Solution for DNN Model via Automated Weight Obfuscation. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 42614–42624. <https://proceedings.mlr.press/v202/zhou23h.html>
- [146] Xirong Zhuang, Lan Zhang, Chen Tang, Huiqi Liu, Bin Wang, Yan Zheng, and Bo Ren. 2023. DeepContract: Controllable Authorization of Deep Learning Models. In *Annual Computer Security Applications Conference, ACSAC 2023, Austin, TX, USA, December 4-8, 2023*. ACM, 609–620. doi:10.1145/3627106.3627107